

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM



BÁO CÁO NGÔN NGỮ LẬP TRÌNH JAVA
XÂY DỰNG HỆ THỐNG CHAT SERVER
ĐA NGƯỜI DÙNG

GVHD: TS. Huỳnh Ngọc Tín

Sinh viên thực hiện:

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

....., ngày.....tháng.....năm 2026

Người nhận xét
(Ký tên và ghi rõ họ tên)

MỤC LỤC

Chương 1: GIỚI THIỆU	6
1.1 Lý do chọn đề tài.....	6
1.2 Mục tiêu của đề tài	7
1.3 Phạm vi đề tài.....	8
1.3.1 Những giới hạn và chức năng hệ thống đã thực hiện	8
1.3.2 Những giới hạn và chức năng chưa thực hiện	9
1.4 Đối tượng sử dụng.....	9
1.4.1 Người dùng thông thường.....	9
1.4.2 Quản trị viên và người điều hành máy chủ trò chuyện.....	9
1.4.3 Quản trị viên hệ thống và bộ phận vận hành	10
Chương 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	10
2.1 Giới thiệu về hệ thống trò chuyện thời gian thực và kiến trúc Microservices phân tán.....	10
2.2 Công nghệ sử dụng cho Backend và Hạ tầng.....	10
2.2.1 Java 17 và Spring Boot	11
2.2.2 Spring Cloud Gateway	11
2.2.3 MySQL và Spring Data JPA.....	11
2.2.4 RabbitMQ và WebSocket	11
2.2.5 MinIO Object Storage.....	11
2.2.6 Prometheus, Grafana và Hệ thống giám sát.....	12
2.2.7 Docker, Jenkins và SonarQube	12
2.3 Công nghệ sử dụng cho Frontend (Client App)	12
2.3.1 Java Swing và FlatLaf.....	12
2.3.2 Jackson và HTTP Client (Java 11+)	12
Chương 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	12
3.1 Yêu cầu chức năng	13
3.1.1 Nhóm chức năng Xác thực và Quản lý hồ sơ (Auth & Profile Services) .	13
3.1.2 Nhóm chức năng Quản lý Mối quan hệ và Bạn bè (Friend Service)	14
3.1.3 Nhóm chức năng Giao tiếp thời gian thực (Messaging Service)	14
3.1.4 Nhóm chức năng Quản lý Không gian trò chuyện (Server & Channel Services).....	15
3.1.5 Nhóm chức năng Phân quyền và Kiểm duyệt (Role Service)	15
3.1.6 Nhóm chức năng Quản lý Tài nguyên Đa phương tiện (File Service)	16

SE330 – Ngôn ngữ lập trình JAVA

3.1.7 Nhóm chức năng Thông báo và Giám trạng thái (Notification & Presence Services).....	16
3.1.8 Nhóm chức năng Ghi vết và Giám sát hệ thống (Log Service)	17
3.2Yêu cầu phi chức năng	17
3.2.1 Tiêu chí Hiệu năng và Độ trễ (Performance & Latency).....	17
3.2.2 Tiêu chí Bảo mật và An toàn thông tin (Security - Defense in Depth).....	17
3.2.3 Tiêu chí Khả năng mở rộng và Tính sẵn sàng (Scalability & Availability)	18
3.2.4 Tiêu chí Khả năng giám sát và Kiểm thử (Observability & Testability) ..	18
3.3Sơ đồ usecase tổng quan và các phân hệ	18
3.3.1 Xác định các Tác nhân (Actors) và Ảnh xạ Hạ tầng 12 Microservices	18
3.3.2 Sơ đồ Use Case Tổng quan Toàn hạ tầng Hệ thống	20
3.3.3 Đặc tả Chi tiết các Nhóm Chức năng Lỗi :	20
3.3.4 Hạ tầng Phân quyền Toán tử Bitmask và Quản trị Quyền hạn Máy chủ (role-service, server-service, channel-service).....	28
3.3.5 Hạ Tầng Hội Thoại Trực Tuyến WebSocket và Truyền Tải Thông điệp Thời Gian Thực (messaging-service, presence-service, gateway-service) ..	33
3.3.6 Hạ Tầng Lưu Trữ Đối Tượng MinIO, Đồng Bộ Thông Báo Đầy và Kiểm Toán Hệ Thống (file-service, notification-service, log-service)	37
3.3.7 Ma Trận Ràng Buộc Kiến Trúc và Chu Trình Phối Hợp Liên Dịch Vụ...	42
Chương 4: THIẾT KẾ HỆ THỐNG	45
4.1 Sơ đồ kiến trúc tổng quan hệ thống:.....	46
4.2Sơ đồ lớp:	46
4.3Sơ đồ tuần tự cho các luồng liên dịch vụ	47
4.4Thiết kế cơ sở dữ liệu chi tiết	47

DANH MỤC BẢNG

Bảng 1: Đặc tả Use Case UC_01 - Đăng ký tài khoản hệ thống	24
Bảng 2: Đặc tả Use Case UC_02 - Đăng nhập hệ thống	24
Bảng 3: Đặc tả Use Case UC_04 - Làm mới phiên làm việc tự động	25
Bảng 4: Đặc tả Use Case UC_08 - Tải lên hình ảnh hồ sơ, server	26
Bảng 5: Đặc tả Use Case UC_11 - Khởi tạo Server mới	27
Bảng 6: Đặc tả Use Case UC_14 - Gia nhập Server bằng mã mời	28
Bảng 7: Đặc tả Use Case UC_18 - Chấp nhận lời mời kết bạn từ hàng đợi	29
Bảng 8: Đặc tả Use Case UC_20 - Khởi tạo vai trò và phân bổ đặc quyền	31
Bảng 9: Đặc tả Use Case UC_22 - Phân bổ và gán vai trò cho hội viên	32
Bảng 10: Đặc tả Use Case UC_23 - Tạo lập cấu trúc kênh văn bản mới	33
Bảng 11: Đặc tả Use Case UC_27 - Cấm thành viên tham gia	34
Bảng 12: Đặc tả Use Case UC_29 - Phát tin nhắn kênh thời gian thực	35
Bảng 13: Đặc tả Use Case UC_31 - Ghim tin nhắn vào danh mục kênh	36
Bảng 14: Đặc tả Use Case UC_33 - Thả biểu cảm tương tác lên tin nhắn	37
Bảng 15: Đặc tả Use Case UC_36 - Tự động ngắt phiên kết nối và chuyển ngoại tuyến	38
Bảng 16: Đặc tả Use Case UC_37 - Tải tệp tin phương tiện lên	40
Bảng 17: Đặc tả Use Case UC_38 - Tự động trích xuất ảnh xem trước	41
Bảng 18: Đặc tả Use Case UC_40 - Tiêu thụ sự kiện hàng đợi và Đẩy thông báo	42
Bảng 19: Đặc tả Use Case UC_43 - Tự động ghi log hệ thống phân tán	43
Bảng 20: Chi tiết ma trận ràng buộc Định danh	45
Bảng 21: Chi tiết ma trận ràng buộc Kiểm soát Đặc quyền	45
Bảng 22: Chi tiết ma trận ràng buộc Nhất quán Hướng sự kiện Bất đồng bộ	46

DANH MỤC HÌNH ẢNH

Hình 1: Sơ đồ Use Case Tổng quan Toàn hạ tầng Hệ thống	22
Hình 2: Sơ đồ usecase Hạ tầng Định danh và Bảo mật	23
Hình 3: Sơ đồ usecase Hạ tầng Server và Bạn bè.....	26
Hình 4: Sơ đồ usecase Hạ tầng Phân quyền và Quản trị	30
Hình 5: Sơ đồ usecase hạ tầng Tín nhắn và Hiện diện	34
Hình 6: Sơ đồ usecase Hạ tầng Lưu trữ, Thông báo.....	39
Hình 7: Chu trình phối hợp các nghiệp vụ.....	43
Hình 8: Sơ đồ kiến trúc tổng quan hệ thống	46
Hình 9: Sơ đồ lớp.....	47
Hình 10: Sơ đồ tuần tự các luồng dịch vụ	47
Hình 11: Cơ sở dữ liệu chat_auth_db và chat_profile_db	48
Hình 12: Cơ sở dữ liệu chat_friend_db và chat_server_db	48
Hình 13: Cơ sở dữ liệu chat_channel_db và chat_role_db.....	49
Hình 14: Cơ sở dữ liệu chat_presence_db và chat_file_db	49
Hình 15: Cơ sở dữ liệu chat_notification_db và chat_messaging_db	49

Chương 1: GIỚI THIỆU

1.1 Lý do chọn đề tài

Trong kỷ nguyên số hóa, nhu cầu trao đổi thông tin đồng bộ thời gian thực (Real-time Communication) ngày càng trở nên phổ biến. Sự phát triển mạnh mẽ của các nền tảng lớn như Discord, Slack hay Microsoft Teams đã thiết lập tiêu chuẩn mới về trải nghiệm người dùng, đòi hỏi hệ thống phải phản hồi với độ trễ cực thấp (Low-latency) và duy trì tính sẵn sàng cao (High Availability) dưới áp lực của hàng triệu thông điệp phân phối mỗi giây.

Việc xây dựng một hệ thống trò chuyện thời gian thực quy mô lớn đặt ra những thách thức kỹ thuật phức tạp:

- **Đặc thù của giao thức truyền thông:** Khác với giao thức HTTP phi trạng thái (Stateless), các ứng dụng thời gian thực yêu cầu duy trì kết nối song công (Full-duplex) dài hạn qua giao thức WebSocket. Chu trình này tiêu tốn lượng lớn tài nguyên phần cứng như bộ nhớ RAM, số lượng tệp tham chiếu (File Descriptors) và luồng xử lý (Threads) khi số lượng người dùng kết nối đồng thời (Concurrent Connections) tăng vọt.
- **Hạn chế của kiến trúc nguyên khối (Monolith):** Việc tích hợp toàn bộ nghiệp vụ vào một khối ứng dụng duy nhất dễ dẫn đến tình trạng cạnh tranh tài nguyên và tạo ra điểm nghẽn hiệu năng (Performance Bottleneck). Các tác vụ tiêu tốn CPU như xác thực bảo mật, hoặc luồng I/O chuyên sâu như truyền tải tệp tin dung lượng lớn sẽ trực tiếp gây đình trệ (Blocking) luồng xử lý của tin nhắn. Đồng thời, một lỗi cục bộ tại bất kỳ phân hệ nào cũng có nguy cơ làm gián đoạn dịch vụ của toàn bộ hệ thống (Single Point of Failure).

Để giải quyết các vấn đề trên, việc chuyển dịch sang kiến trúc **Microservices phân tán** phối hợp với phương pháp Thiết kế hướng miền (Domain-Driven Design) là giải pháp tối ưu. Việc phân rã hệ thống thành các dịch vụ độc lập mang lại hai đặc tính quan trọng: **Cô lập lỗi (Fault Isolation)** và **Khả năng mở rộng độc lập (Independent Scalability)**, giúp khoanh vùng sự cố và tối ưu hóa việc cấp phát tài nguyên hạ tầng một cách chính xác.

Từ nhu cầu thực tiễn đó, đề tài tập trung nghiên cứu và hiện thực hóa một hệ thống phân tán toàn diện bao gồm 12 dịch vụ thành phần phối hợp chặt chẽ:

- **gateway-service:** Đóng vai trò bảo vệ biên, kiểm tra mã Token xác thực trung tâm và định tuyến luồng dữ liệu API.
- **auth-service** và **user-profile-service:** Chuyên trách quản lý định danh mật khẩu bảo mật và dữ liệu hồ sơ cá nhân.
- **server-service**, **channel-service**, và **friend-service:** Tổ chức, điều phối kết cấu Máy chủ cộng đồng, danh mục kênh trò chuyện và mạng lưới bạn bè.
- **role-service:** Thực thi ma trận phân quyền bằng kỹ thuật tính toán Bitmask nhị phân tốc độ cao ở tầng lõi.

- **messaging-service** phối hợp cùng **presence-service**: Duy trì kết nối WebSocket thời gian thực, đồng bộ trạng thái hiện diện và phân phối thông điệp hội thoại qua trung gian Broker RabbitMQ.
- **file-service**: Xử lý độc lập luồng dữ liệu nhị phân với cụm lưu trữ đối tượng MinIO mà không ảnh hưởng đến luồng chat chính.
- **notification-service** và **log-service**: Tiêu thụ sự kiện từ hàng đợi để cập nhật bộ đếm tin nhắn chưa đọc và ghi vết kiểm toán hệ thống bất đồng bộ dưới định dạng JSON Lines.

Vì những lý do kỹ thuật và nhu cầu thực tiễn nêu trên, nhóm quyết định thực hiện đề tài: **"Xây dựng Hệ thống trò chuyện thời gian thực phân tán sử dụng kiến trúc Microservices"**.

1.2 Mục tiêu của đề tài

Mục tiêu cốt lõi của đề tài là nghiên cứu, thiết kế và phát triển thành công một hệ thống trò chuyện trực tuyến có khả năng hoạt động ở quy mô lớn, tuân thủ chặt chẽ mô hình phân tán. Cụ thể:

- Xây dựng hệ thống Backend phân tán bao gồm 12 microservices độc lập (Gateway, Auth, Server, Channel, Messaging, Presence, File, Log, Notification, Role, User Profile, Friend), giao tiếp với nhau qua REST API và Message Broker.
- Triển khai cơ chế xác thực và bảo vệ tuyến đường tập trung tại API Gateway bằng JSON Web Token (JWT).
- Áp dụng mô hình Cơ sở dữ liệu cho từng dịch vụ (Database-per-service pattern) với MySQL nhằm đảm bảo tính lỏng lẻo (loose coupling).
- Phát triển ứng dụng máy trạm (Desktop Client) bằng Java Swing với giao diện hiện đại, hỗ trợ giao tiếp WebSocket thời gian thực và quản lý bộ nhớ đệm hình ảnh hiệu quả.
- Thiết lập quy trình giám sát hệ thống tự động (Monitoring) đo lường hiệu suất của từng microservice và hạ tầng.

1.3 Phạm vi đề tài

Hệ thống "Chat Server Đa Người Dùng" được xây dựng với mục tiêu mô phỏng lại các chức năng cốt lõi của một nền tảng giao tiếp trực tuyến hiện đại (tương tự Discord). Để đảm bảo tính khả thi trong khuôn khổ thời gian của đồ án môn học và tập trung giải quyết triệt để các bài toán về mạng (Networking), đồng thời (Concurrency) và đồng bộ hóa (Synchronization), phạm vi của đề tài được xác định rõ như sau:

1.3.1 Những giới hạn và chức năng hệ thống đã thực hiện

Ứng dụng tập trung hoàn thiện kiến trúc hệ thống phân tán (Microservices) ở phía Backend và giao diện Desktop (Java Swing) ở phía Client, bao gồm các tính năng:

Giao tiếp thời gian thực (Real-time Messaging):

- Xây dựng luồng truyền tải tin nhắn tức thời ứng dụng công nghệ WebSocket.

SE330 – Ngôn ngữ lập trình JAVA

- Hỗ trợ đa dạng hình thức trò chuyện: Chat nhóm theo phòng (Channel) và Chat riêng tư hai người (Direct Message).
- Hỗ trợ các thao tác nâng cao trên tin nhắn: Chỉnh sửa (Edit), Xóa (Soft-delete), Ghim tin nhắn (Pin) và thả cảm xúc (Reaction).

Quản lý không gian trò chuyện:

- Cho phép người dùng tạo các Server riêng, tạo mã mời (Invite Code) để người khác tham gia.
- Tổ chức các phòng chat (Channels) bên trong mỗi Server.

Chia sẻ tài nguyên:

- Hỗ trợ gửi hình ảnh, tài liệu đính kèm qua kênh chat với kích thước tối đa 10MB/file.
- Hệ thống tự động xử lý, tạo ảnh thu nhỏ (thumbnail) để tối ưu băng thông hiển thị cho Client.

Quản lý người dùng & Trạng thái:

- Đồng bộ trạng thái hoạt động theo thời gian thực (Online, Idle, Do Not Disturb, Offline/Invisible).
- Gửi yêu cầu kết bạn, hiển thị danh sách bạn bè trực tuyến.

Phân quyền và bảo mật cơ bản:

- Xác thực người dùng qua cơ chế JWT Token an toàn.
- Quản lý vai trò (Role-based Access Control) trong Server với hệ thống Permission Bitmask (quyền Owner, Admin, Moderator), hỗ trợ Kick/Ban thành viên.

Giao diện người dùng (GUI): Xây dựng ứng dụng Client dành cho Desktop PC bằng Java Swing (sử dụng thư viện FlatLaf), hỗ trợ chế độ Light/Dark mode và tùy biến ảnh nền.

1.3.2 Những giới hạn và chức năng chưa thực hiện

Để tập trung tối ưu hóa hiệu suất xử lý đa luồng và luồng tin nhắn văn bản/file, một số tính năng nâng cao nằm ngoài phạm vi thực hiện của đề án này:

Không hỗ trợ Gọi thoại và Gọi video: Mặc dù hệ thống có thiết kế sẵn định dạng phân loại ChannelType.VOICE trong cơ sở dữ liệu, nhưng đề án chưa tích hợp các giao thức truyền phát đa phương tiện (WebRTC, UDP Streaming). Ứng dụng hiện tại giới hạn hoàn toàn ở việc giao tiếp bằng văn bản (Text) và Tập tin (Files).

Không hỗ trợ truyền tải tập tin dung lượng lớn (P2P File Transfer): Để đảm bảo an toàn cho máy chủ lưu trữ (MinIO), hệ thống giới hạn cứng kích thước upload là 10MB/file. Đề tài không giải quyết bài toán truyền file Peer-to-Peer hay cơ chế Resume (tiếp tục tải) khi kết nối mạng bị ngắt.

Chưa có Mã hóa đầu cuối (End-to-End Encryption - E2EE): Tin nhắn được truyền tải an toàn giữa Client và Server qua JWT, nhưng nội dung tin nhắn vẫn được lưu trữ dưới dạng bản rõ (plaintext) trong cơ sở dữ liệu để phục vụ chức năng tìm kiếm toàn văn. Chức năng mã hóa E2EE tại client không thuộc phạm vi đề án.

Nền tảng hỗ trợ (Platforms): Giao diện frontend hiện tại chỉ hoạt động trên môi trường Desktop PC cài đặt Java Runtime (Windows/macOS/Linux). Đề tài chưa phát triển các phiên bản chạy trên trình duyệt Web (Web Client) hay ứng dụng di động (Mobile App).

1.4 Đối tượng sử dụng

Hệ thống được thiết kế và phân cấp thành ba nhóm đối tượng sử dụng chính, tương ứng với từng mức độ phân quyền và mục đích tương tác cụ thể:

1.4.1 Người dùng thông thường

Đặc điểm: Đây là tệp người dùng cuối trực tiếp tương tác với hệ thống thông qua giao diện đồ họa trên máy tính.

Mục đích sử dụng: Sử dụng ứng dụng làm công cụ giao tiếp, trò chuyện trực tuyến, quản lý các mối quan hệ (kết bạn) và chia sẻ thông tin.

Quyền hạn và chức năng: Được cấp các đặc quyền cơ bản như đăng ký, đăng nhập an toàn, cập nhật hồ sơ cá nhân (đổi tên hiển thị, ảnh đại diện, trạng thái hoạt động). Nhóm người dùng này có thể tạo các cuộc hội thoại riêng tư (nhắn tin 1-1), khởi tạo máy chủ trò chuyện cá nhân hoặc tham gia vào các không gian cộng đồng thông qua mã mời.

1.4.2 Quản trị viên và người điều hành máy chủ trò chuyện

Đặc điểm: Là những người dùng thông thường nhưng được cấp thêm các đặc quyền quản lý cấp cao trong phạm vi một máy chủ trò chuyện (Server) cụ thể. Hệ thống quản lý nhóm này thông qua cơ chế phân quyền linh hoạt bằng mặt nạ bit (permission bitmask).

Mục đích sử dụng: Duy trì trật tự, quản lý không gian giao tiếp và điều phối các thành viên bên trong máy chủ.

Quyền hạn và chức năng: Nhóm này được chia thành các cấp bậc như Chủ sở hữu (người tạo ra máy chủ), Quản trị viên và Người điều hành. Tùy vào phân quyền, họ có thể thực hiện các nghiệp vụ:

Khởi tạo, chỉnh sửa hoặc xóa bỏ các kênh giao tiếp (bằng văn bản hoặc đa phương tiện).

Thiết lập và phân phát vai trò cho các thành viên khác.

Kiểm duyệt nội dung: gỡ bỏ tin nhắn vi phạm, mời xuất hoặc cấm vĩnh viễn các thành viên có hành vi không phù hợp ra khỏi máy chủ.

1.4.3 Quản trị viên hệ thống và bộ phận vận hành

Đặc điểm: Đây là đội ngũ kỹ sư phần mềm, người triển khai và bảo trì dự án. Nhóm này không tương tác qua giao diện trò chuyện thông thường mà làm việc trực tiếp với kiến trúc vi dịch vụ ở phía máy chủ.

Mục đích sử dụng: Đảm bảo hệ thống hoạt động ổn định, trơn tru, giám sát lưu lượng thực tế và khắc phục các sự cố về mạng hoặc quá tải bộ nhớ.

Quyền hạn và chức năng: Thao tác trên các hệ thống quản trị nội bộ độc lập, bao gồm: Giám sát hiệu năng phần cứng, mức tiêu thụ tài nguyên và lưu lượng mạng qua các bảng điều khiển trực quan (Grafana và Prometheus).

Quản lý và theo dõi các luồng tin nhắn đang được vận chuyển trong hệ thống hàng đợi (RabbitMQ).

Kiểm soát không gian lưu trữ tài nguyên đa phương tiện chuyên biệt (MinIO).

Truy xuất, phân tích nhật ký hoạt động hệ thống và truy vết lỗi thông qua dịch vụ lưu trữ nhật ký tập trung.

Chương 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Giới thiệu về hệ thống trò chuyện thời gian thực và kiến trúc Microservices phân tán

Trái ngược với các ứng dụng web tĩnh truyền thống sử dụng giao thức HTTP yêu cầu phản hồi (request-response), hệ thống trò chuyện đòi hỏi kết nối hai chiều liên tục (full-duplex) thông qua giao thức WebSocket để đẩy tin nhắn ngay lập tức đến máy khách. Để đối phó với số lượng kết nối duy trì liên tục (persistent connections) cực kỳ lớn, kiến trúc Microservices phân tách ứng dụng thành nhiều dịch vụ nhỏ. Nhờ vậy, bài toán chịu tải được phân tán: messaging-service chỉ tập trung duy trì kết nối WebSocket và đẩy tin nhắn; file-service độc lập xử lý stream tệp tin lớn mà không chặn luồng (blocking) hệ thống; và presence-service quản lý trạng thái online/offline bằng cách nhận sự kiện từ các dịch vụ khác. Việc cô lập này đảm bảo nếu dịch vụ tải tệp tin gặp sự cố, người dùng vẫn có thể gửi và nhận tin nhắn văn bản bình thường.

2.2 Công nghệ sử dụng cho Backend và Hạ tầng

2.2.1 Java 17 và Spring Boot

Ngôn ngữ Java 17 kết hợp cùng bộ khung Spring Boot (phiên bản 3.x) là nền tảng cốt lõi được sử dụng để phát triển toàn bộ 12 dịch vụ backend. Spring Boot loại bỏ sự phức tạp của việc cấu hình các thành phần ứng dụng, cung cấp môi trường nhúng (embedded web server) mạnh mẽ. Việc giao tiếp nội bộ giữa các microservices (Inter-service communication) được thực hiện linh hoạt thông qua thư viện Spring Cloud OpenFeign, cho phép một dịch vụ gọi REST API của dịch vụ khác như các lời gọi hàm thông thường.

2.2.2 Spring Cloud Gateway

Trong kiến trúc phân tán, thay vì để Client kết nối trực tiếp với 12 dịch vụ ở 12 cổng (port) khác nhau, hệ thống sử dụng Spring Cloud Gateway (gateway-service) chạy ở cổng 8080 làm điểm truy cập duy nhất. Gateway đóng vai trò định tuyến (routing) các API request đến đúng dịch vụ đích. Đặc biệt, Gateway tích hợp JwtAuthFilter nhằm bóc tách và phân giải chuỗi xác thực JSON Web Token (JWT) ngay tại cổng vào, bảo vệ các tuyến đường nội bộ khỏi truy cập trái phép và giảm tải việc xác thực cho từng dịch vụ lẻ.

2.2.3 MySQL và Spring Data JPA

Để tuân thủ triệt để mô hình độc lập dữ liệu (Database-per-service), hệ thống triển khai một MySQL Server với 12 logical databases khác biệt (ví dụ: chat_auth_db, chat_messaging_db, chat_file_db...). Spring Data JPA được sử dụng để ánh xạ đối tượng-quan hệ (ORM) bằng Hibernate, giúp thực thi các thao tác CRUD và thiết lập các ràng buộc dữ liệu (indexes, unique constraints) một cách tự động và nhất quán trong mã nguồn Java.

2.2.4 RabbitMQ và WebSocket

- **WebSocket:** Được cấu hình trong messaging-service thông qua WebSocketConfigurer, cung cấp endpoint /ws/chat để Desktop Client thiết lập kết nối thời gian thực.
- **RabbitMQ:** Đóng vai trò là Trạm trung chuyển thông điệp (Message Broker) ứng dụng Kiến trúc hướng sự kiện (Event-Driven). RabbitMQ thiết lập định tuyến chat.exchange theo dạng Topic Exchange. Khi một tin nhắn được gửi, nó không chỉ được đẩy qua WebSocket mà còn được phát sóng (publish) qua RabbitMQ. Từ đó, log-service (lắng nghe log.#) và notification-service (lắng nghe notify.#) tiêu thụ thông điệp một cách bất đồng bộ để ghi chép hoặc phân tích lượt đề cập (mentions) mà không làm chậm trễ độ phản hồi của hệ thống chat chính.

2.2.5 MinIO Object Storage

Thay vì lưu trữ hình ảnh và tài liệu dạng nhị phân trực tiếp vào MySQL (gây quá tải DB), hệ thống sử dụng MinIO – một máy chủ lưu trữ tương thích Amazon S3. Dịch vụ file-service sử dụng MinioClient để thao tác trực tiếp với MinIO nhằm tải lên, tải xuống (upload/download) tài nguyên giới hạn ở 10MB. Bên cạnh đó, hệ thống ứng dụng thư viện Thumbnailator để tự động tạo ra hình thu nhỏ (thumbnail) kích thước 200x200 pixels cho tệp tin hình ảnh, tối ưu hóa băng thông tải dữ liệu cho Client.

2.2.6 Prometheus, Grafana và Hệ thống giám sát

Mỗi Spring Boot microservice đều kích hoạt spring-boot-starter-actuator để bộc lộ điểm cuối giám sát sức khỏe và số liệu thống kê /actuator/prometheus. Máy chủ Prometheus được cấu hình (prometheus.yml) để thu thập dữ liệu (scrape) định kỳ từ 12 dịch vụ theo chu kỳ 10 giây. Grafana đóng vai trò trực quan hóa các dữ liệu số liệu này thành bảng điều khiển (Dashboards), giúp quản trị viên nắm bắt nhanh chóng trạng thái và độ trễ của hệ thống.

2.2.7 Docker, Jenkins và SonarQube

Hệ thống tận dụng Docker và Docker Compose để đóng gói toàn bộ vòng đời ứng dụng. docker-compose.yml quy định cách thức vận hành các thùng chứa (containers) của MySQL, RabbitMQ, MinIO và 12 microservices trên chung một mạng nội bộ ảo chat-net. Đồng thời, trong môi trường DevOps, Jenkins phối hợp cùng Jenkinsfile để tự động hóa đường ống CI/CD, song hành với SonarQube tĩnh phân tích chất lượng, bảo mật mã nguồn.

2.3 Công nghệ sử dụng cho Frontend (Client App)

2.3.1 Java Swing và FlatLaf

Frontend của hệ thống được xây dựng dưới dạng Desktop Application thuần túy sử dụng Java Swing. Mặc dù Swing là một công nghệ có tuổi đời lâu năm, hệ thống đã ứng dụng giao diện **FlatLaf (Flat Look and Feel)** để hiện đại hóa triệt để trải nghiệm người dùng (UX/UI), cung cấp giao diện phẳng, mượt mà với các khả năng tùy chỉnh màu sắc theo

chủ đề Sáng/Tối (Light/Dark Mode). Đặc biệt, thay vì phụ thuộc vào thư viện hình ảnh nặng nề, các biểu tượng (Icons) trong ứng dụng đều được vẽ động (Vector) bằng thư viện Java2D (Graphics2D, RenderingHints.KEY_ANTIALIASING), đảm bảo độ sắc nét trên mọi độ phân giải màn hình.

2.3.2 Jackson và HTTP Client (Java 11+)

Lớp mạng (Network Layer) của Client ứng dụng gói API tiêu chuẩn `java.net.http.HttpClient` (ra mắt từ Java 11) để gọi các điểm cuối RESTful bất đồng bộ và mở luồng kết nối WebSocket với máy chủ. Thư viện **Jackson** (ObjectMapper) được sử dụng xuyên suốt như bộ máy phân tích cú pháp JSON, chuyển đổi các gói phản hồi từ backend (như MessageDTO) thành đối tượng Java dễ dàng quản lý thông qua lớp kết nối tùy chỉnh `JsonMapper` hỗ trợ định dạng thời gian `JavaTimeModule`.

Chương 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu chức năng

Hệ thống được thiết kế theo kiến trúc phân tán hướng dịch vụ (Microservices), phân tách dữ liệu theo mô hình biệt lập cơ sở dữ liệu (Database-per-Service) nhằm triệt tiêu sự phụ thuộc và tối ưu hóa khả năng phân phối tải. Dựa trên cấu trúc triển khai thực tế của các phân hệ dịch vụ độc lập, tập hợp yêu cầu chức năng của toàn hệ thống được phân định chi tiết như sau:

3.1.1 Nhóm chức năng Xác thực và Quản lý hồ sơ (Auth & Profile Services)

Nhóm nghiệp vụ này chịu trách nhiệm kiểm soát toàn bộ vòng đời tài khoản định danh, thiết lập cơ chế bảo mật biên và cung cấp các tính năng cá nhân hóa trải nghiệm người dùng cuối:

- **Đăng ký tài khoản:** Hệ thống cho phép người dùng khởi tạo tài khoản mới thông qua việc kiểm tra tính duy nhất của tên tài khoản (username) trong hệ thống cơ sở dữ liệu `chat_auth_db`. Mật khẩu người dùng được mã hóa một chiều bằng thuật toán BCrypt cường độ cao trước khi lưu trữ xuống đĩa cứng.
- **Đăng nhập và Duy trì phiên làm việc:** Xác thực thông tin người dùng dựa trên cơ chế đối khớp chuỗi băm của Spring Security. Khi xác thực thành công, dịch vụ phát hành một cặp mã thông báo bảo mật bao gồm: một mã truy cập ngắn hạn (Access Token ký bằng thuật toán bảo mật HMAC-SHA) có thời hạn hiệu lực cụ thể và một mã làm mới dài hạn (Refresh Token) định dạng chuỗi UUID ngẫu nhiên.
- **Cơ chế tự động làm mới phiên làm việc (Refresh Token):** Nhằm tối ưu hóa trải nghiệm người dùng và ngăn chặn tình trạng ngắt kết nối đột ngột, ứng dụng Client cài đặt luồng interceptor tự động bắt lỗi hết hạn token. Hệ thống gửi yêu cầu không đồng bộ mang theo Refresh Token đến điểm cuối `/api/auth/refresh` để cấp phát một Access Token hoàn toàn mới mà không yêu cầu người dùng tái nhập mật khẩu.

- **Đổi mật khẩu bảo mật:** Người dùng thiết lập lại chuỗi mật khẩu định kỳ. Hệ thống yêu cầu xác thực mật khẩu cũ thông qua bộ lọc bảo mật và thực hiện cập nhật lại chuỗi băm mới trực tiếp vào thực thể người dùng.
- **Cập nhật và Cá nhân hóa hồ sơ:** Người dùng có quyền tùy biến thông tin hiển thị bao gồm tên gọi đại diện (displayName) và đoạn tiểu sử cá nhân (bio) với độ dài giới hạn trong phạm vi \$500\$ ký tự.
- **Tải lên và Quản lý ảnh đại diện (Avatar):** Hệ thống cho phép người dùng tải tệp ảnh định dạng JPEG hoặc PNG thông qua luồng dữ liệu Multipart. Phân hệ quản lý tệp áp đặt cấu hình kiểm soát dung lượng pay-load nghiêm ngặt, từ chối xử lý và trả về lỗi hệ thống MaxUploadSizeExceededException nếu dung lượng tệp vượt quá ngưỡng 2MB.
- **Thiết lập trạng thái hoạt động trực tuyến:** Tích hợp chặt chẽ với dịch vụ quản lý trạng thái (presence-service), cho phép người dùng thiết lập tường minh các chế độ hiển thị bao gồm: Trực tuyến (ONLINE), Không hoạt động (IDLE/AWAY), Không làm phiền (DO_NOT_DISTURB), và Ẩn mình/Ngoại tuyến (INVISIBLE/OFFLINE).
- **Tra cứu và Tìm kiếm người dùng:** Cung cấp khả năng tìm kiếm người dùng khác dựa trên cơ chế quét khớp chuỗi không phân biệt hoa thường (ContainingIgnoreCase) đối với trường username và displayName trong cơ sở dữ liệu, tự động loại trừ chính tài khoản đang thực hiện lệnh tra cứu.

3.1.2 Nhóm chức năng Quản lý Mối quan hệ và Bạn bè (Friend Service)

Hỗ trợ người dùng xây dựng và điều hành mạng lưới liên kết xã hội cá nhân hóa, hoạt động độc lập cấu trúc phân cấp của các máy chủ cộng đồng:

- **Quản lý vòng đời lời mời kết bạn:** Người dùng phát đi yêu cầu kết bạn đến một người dùng khác sau khi hệ thống gọi liên dịch vụ (Inter-service Communication) sang user-profile-service để xác minh sự tồn tại của tài khoản mục tiêu. Trạng thái lời mời được thiết lập ban đầu là PENDING.
- **Phản hồi yêu cầu kết bạn:** Người nhận có quyền duyệt danh sách lời mời đang chờ xử lý, đưa ra quyết định Chấp nhận (ACCEPTED) để chính thức thiết lập mối quan hệ bạn bè hai chiều, hoặc Hủy bỏ/Từ chối (REJECT) để xóa bỏ bản ghi liên kết khỏi bảng dữ liệu friendships.
- **Chặn liên lạc cá nhân (Block):** Cho phép chuyển đổi trạng thái mối quan hệ sang BLOCKED, cắt đứt toàn bộ khả năng tương tác trực tiếp và tìm kiếm từ phía tài khoản bị chặn nhằm bảo vệ quyền riêng tư.

3.1.3 Nhóm chức năng Giao tiếp thời gian thực (Messaging Service)

Đây là cụm chức năng lõi của hệ thống, vận hành trên nền tảng kết nối Socket bền vững duy trì bởi ChatWebSocketHandler kết hợp cùng mạng lưới điều phối thông điệp bất đồng bộ của Message Broker RabbitMQ:

- **Nhắn tin riêng tư kênh đôi (Direct Message 1-1):** Xử lý luồng hội thoại bảo mật giữa hai người dùng. Tin nhắn được lưu trữ vào cơ sở dữ liệu tập trung chat_messaging_db và phân phối tức thời thông qua việc định tuyến phiên làm việc cục bộ của tác nhân nhận.

- **Nhắn tin nhóm trong Kênh máy chủ (Channel Chat):** Điều phối toàn bộ luồng hội thoại văn bản đa người dùng trong một phân hệ kênh thảo luận. Khi tin nhắn được gửi, hệ thống lưu trữ đồng thời đẩy thông điệp lên Fanout Exchange của RabbitMQ (chat.fanout) để thực hiện kỹ thuật phát quảng bá (Broadcast) đến mọi node WebSocket đang hoạt động, cập nhật giao diện thời gian thực cho toàn bộ thành viên thuộc máy chủ.
- **Chỉnh sửa nội dung thông điệp (Message Edit):** Người dùng có quyền sửa đổi nội dung tin nhắn do chính mình phát đi. Hệ thống kiểm tra quyền sở hữu (isMessageOwner), cập nhật lại nội dung trong cơ sở dữ liệu, bật cờ đánh dấu isEdited = true và phát quảng bá sự kiện EDIT để cập nhật trực tiếp lên giao diện của các client khác.
- **Thu hồi thông điệp (Message Recall/Delete):** Hỗ trợ cơ chế xóa mềm tin nhắn (Soft Delete). Khi lệnh gỡ được thực thi bởi người gửi hoặc ban quản trị có thẩm quyền, nội dung tin nhắn lập tức được chuyển thành chuỗi hằng số hệ thống "Tin nhắn bị gỡ", đồng thời hệ thống tự động gọi REST API sang channel-service để gỡ bỏ trạng thái ghim nếu tin nhắn đó đang được lưu giữ trong danh sách ghim.
- **Lưu giữ tin nhắn quan trọng (Pin Message):** Thành viên có thẩm quyền có quyền ghim các thông điệp có giá trị lên phân hệ lưu trữ của kênh (pinned_messages). Hệ thống lưu mapping giữa channelId và messageId kèm thông tin người ghim, cho phép toàn bộ thành viên truy xuất nhanh danh sách thông báo qua một giao diện hộp thoại tập trung.
- **Tương tác cảm xúc (Message Reaction):** Người dùng tương tác động bằng cách thả các biểu tượng cảm xúc lên từng bong bóng tin nhắn cụ thể. Hệ thống quản lý thực thể MessageReaction theo cơ chế Idempotent: nếu người dùng thả trùng một biểu tượng cảm xúc, hệ thống tự động hiểu là hành vi hủy bỏ cảm xúc đó (REMOVE), ngược lại sẽ tiến hành thêm mới (ADD) và cập nhật số lượng kiểm đếm thời gian thực.
- **Tra cứu lịch sử hội thoại toàn văn:** Cung cấp bộ lọc tìm kiếm tin nhắn theo từ khóa theo ba cấp độ phạm vi linh hoạt: Tìm kiếm cô lập bên trong một kênh văn bản (searchInChannel), tra cứu trong luồng chat riêng tư (searchInPrivate), hoặc quét toàn cục trên mọi không gian hội thoại mà người dùng tham gia (searchAllForUser).

3.1.4 Nhóm chức năng Quản lý Không gian trò chuyện (Server & Channel Services)

Cung cấp các công cụ cấu trúc hóa và quản trị các không gian cộng đồng quy mô lớn:

- **Khởi tạo và Quản trị Máy chủ (Server):** Người dùng có quyền khởi tạo một không gian máy chủ trò chuyện mới và tự động trở thành Chủ máy chủ (Server Owner). Hệ thống tự động phát sinh một chuỗi mã mời (inviteCode) ngẫu nhiên có độ dài \$8\$ ký tự bằng thuật toán cắt chuỗi UUID và tự động kiến tạo một kênh văn bản hệ thống mặc định mang tên "General".
- **Điều hành thành viên và Mã mời:** Người dùng tham gia vào máy chủ bằng cách nhập mã mời trực tiếp vào ô điều hướng. Chủ máy chủ có quyền làm mới chuỗi mã mời này bất kỳ lúc nào để vô hiệu hóa các đường liên kết cũ.

- **Tổ chức phân cấp Kênh giao tiếp (Channel):** Cho phép chia nhỏ máy chủ thành các không gian chức năng chuyên biệt bao gồm Kênh văn bản (TEXT) phục vụ nhắn tin và Kênh đàm thoại (VOICE). Hỗ trợ thiết lập chủ đề kênh (topic), chế độ giới hạn tốc độ gửi tin (slowmode tính bằng giây) và gom nhóm các kênh theo danh mục phân loại (category).

3.1.5 Nhóm chức năng Phân quyền và Kiểm duyệt (Role Service)

Vận hành cơ chế kiểm soát truy cập và bảo vệ an ninh không gian cộng đồng dựa trên nền tảng kiến trúc phân quyền theo vai trò (Role-Based Access Control - RBAC):

- **Mô hình hóa quyền hạn bằng Mặt nạ bit (Permission Bitmask):** Hệ thống định nghĩa các quyền hạn cốt lõi dưới dạng các bit độc lập dịch chuyển nhị phân (ví dụ: $\text{SEND_MESSAGE} = 1 \ll 0$, $\text{READ_MESSAGES} = 1 \ll 1$, $\text{MANAGE_MESSAGES} = 1 \ll 2$, v.v.). Toàn bộ quyền hạn của một Vai trò (Role) được gộp lại thành một số nguyên duy nhất (permissionBitmask) thông qua phép toán logic OR nhị phân, giúp tối ưu hóa không gian lưu trữ cấu trúc quyền và tăng tốc độ tính toán kiểm tra quyền tại lớp lọc API Gateway.
- **Quản lý và Phân tầng Vai trò:** Ban quản trị có quyền khởi tạo các vai trò tùy chỉnh, thiết lập mã màu đại diện dạng Hex và gán thứ tự ưu tiên hiển thị (priority). Khi một máy chủ được khởi tạo, hệ thống tự động thiết lập các vai trò mặc định không thể xóa bao gồm: Owner (đầy đủ mọi quyền, giá trị bitmask tối đa là \$255\$), Admin (\$127\$), Moderator (\$23\$) và Member (\$3\$).
- **Thi hành chế tài kiểm duyệt:** Cấp thẩm quyền kiểm duyệt có quyền trục xuất thành viên vi phạm pháp quy ra khỏi máy chủ (Kick - thành viên có thể vào lại nếu có mã mời) hoặc thực thi lệnh cấm triệt để (Ban - ghi dữ liệu vĩnh viễn vào bảng banned_members, chặn hoàn toàn khả năng sử dụng mã mời để gia nhập lại cho đến khi được gỡ cấm).

3.1.6 Nhóm chức năng Quản lý Tài nguyên Đa phương tiện (File Service)

Đảm bảo khả năng truyền tải, phân phối và tổ chức lưu trữ các tệp tin tài nguyên an toàn thông qua việc tích hợp hệ thống lưu trữ đối tượng tương thích chuẩn S3 (MinIO):

- **Tải lên tệp đính kèm đa định dạng:** Người dùng chia sẻ các tệp tin bao gồm hình ảnh, video nhỏ, tài liệu văn bản (PDF, Word) và mã nguồn vào luồng chat. Hệ thống áp đặt bộ lọc kiểm tra ở tầng nghiệp vụ, áp ngưỡng dung lượng tối đa cho mỗi lượt tải lên là 10MB và chặn toàn bộ các tệp tin thực thi nguy hiểm (như .exe, .bat) nhằm phòng chống các cuộc tấn công mã độc.
- **Tự động khởi tạo ảnh thu nhỏ (Thumbnail):** Khi phát hiện tệp tin tải lên thuộc nhóm định dạng hình ảnh, hệ thống kích hoạt không đồng bộ dịch vụ ThumbnailService. Sử dụng thư viện đồ họa để nén cấu trúc ảnh, đưa kích thước độ phân giải về tỷ lệ chuẩn \$200 \times 200\$ pixel định dạng JPEG nhằm tối ưu hóa băng thông truyền tải và tăng tốc độ render giao diện lưới của phía Client.
- **Thư viện tài nguyên kênh (Media Gallery & File List):** Tự động phân loại và lập chỉ mục toàn bộ các tệp tin đã chia sẻ trong một kênh thành hai kho lưu trữ trực quan: phân hệ Ảnh/Video (image) và phân hệ Tài liệu (document), giúp các thành viên dễ dàng tra cứu lại thông qua thanh sidebar tiện ích.

3.1.7 Nhóm chức năng Thông báo và Giám trạng thái (Notification & Presence Services)

Duy trì tính liên tục của luồng thông tin và điều phối các sự kiện cảnh báo tương tác của người dùng cuối:

- **Đồng bộ trạng thái trực tuyến thời gian thực:** Giám sát liên tục trạng thái kết nối WebSocket của người dùng. Khi xảy ra sự kiện kết nối (connect) hoặc ngắt kết nối (disconnect), hệ thống cập nhật bộ nhớ đệm và phát sự kiện STATUS qua hàng đợi RabbitMQ để đồng bộ hóa lập tức giao diện hiển thị danh sách bạn bè và thành viên trên toàn bộ các client đang trực tuyến.
- **Phân tích cú pháp Nhắc tên (Mention Detection):** Khi nhận một tin nhắn mới từ hàng đợi, dịch vụ thông báo tiến hành quét nội dung bằng biểu thức chính quy (Regex) `@(\w+)`. Nếu phát hiện hành vi nhắc tên cá nhân (`@username`) hoặc nhắc toàn thể nhóm (`@everyone`), hệ thống sẽ khởi tạo một thực thể thông báo Notification thuộc loại tương ứng lưu vào cơ sở dữ liệu và kích hoạt lệnh đẩy thông báo nổi (OS Notification) đến thiết bị của người nhận.
- **Kiểm đếm và Quản lý bộ đếm chưa đọc (Unread Counter):** Tự động tính toán lũy tiến số lượng tin nhắn chưa đọc của từng người dùng đối với từng kênh văn bản hoặc cuộc trò chuyện riêng tư. Bộ đếm này được hiển thị dưới dạng huy hiệu (badge) nổi trên giao diện Client và sẽ lập tức được đặt về giá trị \$0\$ (Reset) khi người dùng gửi lệnh xác nhận đã đọc (Acknowledge/Ack).

3.1.8 Nhóm chức năng Ghi vết và Giám sát hệ thống (Log Service)

Đây là cụm chức năng ngầm (Background Service) phục vụ riêng cho mục đích quản trị hạ tầng, vận hành theo kiến trúc hướng sự kiện (Event-Driven Architecture):

- **Lắng nghe sự kiện hệ thống bất đồng bộ:** Dịch vụ LogEventListener đăng ký xếp hàng thụ động trên queue chat.log.queue của RabbitMQ để hứng toàn bộ các payload sự kiện phát ra từ luồng nghiệp vụ của hệ thống (bao gồm các hành vi như đăng nhập, đăng ký, gửi tin nhắn, sửa/xóa dữ liệu).
- **Lưu trữ nhật ký cấu trúc dữ liệu:** Toàn bộ dữ liệu sự kiện được chuyển đổi thành cấu trúc đối tượng LogEntry, định dạng hóa thành chuỗi văn bản JSON Lines (mỗi dòng là một bản ghi JSON độc lập) và thực hiện ghi nối đuôi (Append) xuống tệp tin nhật ký hệ thống chat_log.txt. Cơ chế này đảm bảo tính thread-safe tuyệt đối thông qua kỹ thuật đồng bộ khóa ghi writeLock, phục vụ hiệu quả cho công tác kiểm toán (Audit) và truy vết lỗi (Troubleshooting) hạ tầng.

3.2 Yêu cầu phi chức năng

Bên cạnh các năng lực xử lý nghiệp vụ trực quan, hệ thống phải đáp ứng các tiêu chuẩn nghiêm ngặt về mặt kiến trúc phần mềm, tính ổn định hạ tầng và các chỉ số an toàn thông tin của một nền tảng giao tiếp phân tán:

3.2.1 Tiêu chí Hiệu năng và Độ trễ (Performance & Latency)

- **Độ trễ truyền tải thời gian thực:** Luồng phân phối thông điệp chat thông qua giao thức kế thừa WebSocket và hàng đợi trung chuyển RabbitMQ phải bảo đảm

thời gian phản hồi (End-to-End Latency) dưới ngưỡng 200ms trong điều kiện vận hành mạng thông thường.

- **Tối ưu hóa tài nguyên JVM:** Các vi dịch vụ Java Backend được thiết lập cấu hình tinh chỉnh bộ thu gom rác tối ưu cho môi trường container (-XX:+UseSerialGC -Xmx256m/-Xmx512m) nhằm giảm thiểu xung đột tài nguyên, triệt tiêu hiện tượng thắt nút cổ chai (Stop-the-world) và bảo đảm tính ổn định khi triển khai đồng thời nhiều container trên cùng một máy chủ vật lý.

3.2.2 Tiêu chí Bảo mật và An toàn thông tin (Security - Defense in Depth)

- **Xác thực phi tập trung biên:** Lớp API Gateway (gateway-service) đóng vai trò là chốt chặn bảo mật duy nhất, chịu trách nhiệm chặn lọc toàn bộ các request không hợp lệ. Sử dụng bộ lọc toàn cục JwtAuthFilter để giải mã chữ ký số của JWT Token bằng khóa bí mật (jwt.secret), sau đó thực hiện kỹ thuật đột biến request (Mutate Request) để bơm thông tin định danh X-User-Id vào Header trước khi chuyển tiếp vào mạng nội bộ.
- **Chiến lược phòng thủ chiều sâu (Defense-in-Depth):** Để ngăn chặn triệt để lỗ hổng rò rỉ dữ liệu hoặc tấn công SSRF thông qua việc Client tải các tài nguyên từ máy chủ độc hại bên ngoài, hệ thống cài đặt lớp ràng buộc nghiêm ngặt UrlUtils tại tầng Service của user-profile-service và server-service. Hệ thống từ chối lưu trữ các URL tuyệt đối hướng tới host lạ, chỉ chấp nhận lưu trữ và phân phối các đường dẫn tương đối nội bộ dạng /api/files/... hoặc /api/users/avatars/....

3.2.3 Tiêu chí Khả năng mở rộng và Tính sẵn sàng (Scalability & Availability)

- **Kiến trúc dữ liệu phân tán độc lập:** Áp dụng triệt để nguyên lý thiết kế Microservices, mỗi dịch vụ sở hữu hoàn toàn một schema cơ sở dữ liệu riêng biệt (chat_auth_db, chat_channel_db, chat_messaging_db, chat_role_db, v.v.). Thiết kế này cho phép bất kỳ một dịch vụ nào gặp sự cố (ví dụ: file-service quá tải) thì toàn bộ các chức năng cốt lõi khác như xác thực hay nhắn tin văn bản vẫn vận hành bình thường.
- **Đóng gói và Hạ tầng hóa đồng bộ:** Toàn bộ hệ thống Backend được đóng gói thành các Docker Image tiêu chuẩn dựa trên phôi base-image ổn định eclipse-temurin:17-jre-jammy, cho phép triển khai nhanh chóng, nhất quán trên mọi môi trường từ Local Development cho đến môi trường Production thông qua tệp điều phối docker-compose.yml.

3.2.4 Tiêu chí Khả năng giám sát và Kiểm thử (Observability & Testability)

- **Thu thập số liệu đo lường hạ tầng (Metrics Exporter):** Tất cả các phân hệ dịch vụ Java Spring Boot đều được tích hợp module spring-boot-starter-actuator kết hợp cùng thư viện micrometer-registry-prometheus. Cấu hình hệ thống mở khai báo công khai các endpoint /actuator/prometheus cho phép máy chủ giám sát Prometheus chủ động cào (scrape) dữ liệu về hiệu năng hệ thống (lưu lượng CPU, RAM, số lượng connection, tần suất request) theo chu kỳ định sẵn và hiển thị trực quan lên dashboard Grafana.

3.3 Sơ đồ usecase tổng quan và các phân hệ

3.3.1 Xác định các Tác nhân (Actors) và Ảnh xạ Hạ tầng 12 Microservices

Để đặc tả toàn diện một hệ thống phân tán bao gồm 12 dịch vụ nền tảng phối hợp thông qua cơ chế bất đồng bộ, việc xác định tác nhân không chỉ đơn thuần dừng lại ở góc độ người dùng cuối, mà phải bóc tách sâu vào mối quan hệ tương tác với mã nguồn lỗi, hạ tầng lưu trữ đối tượng và hàng đợi tin nhắn. Hệ thống phân định rõ ràng 5 nhóm tác nhân chính sau:

1. Người dùng phổ thông (Regular User)

Tác nhân đại diện cho thực thể người dùng sau khi vượt qua chốt chặn định danh trung tâm. Tác nhân này tương tác trực tiếp với các dịch vụ nền tảng tuyến đầu bao gồm:

- gateway-service: Nơi tiếp nhận các yêu cầu HTTP/giao thức kết nối, thực hiện bộ lọc toàn cục JwtAuthFilter để bóc tách luồng Token, xác thực chữ ký bảo mật và tự động bơm các thông tin định danh X-User-Id cùng X-Username vào tiêu đề Header của gói tin trước khi chuyển tiếp downstream.
- auth-service: Chịu trách nhiệm xử lý các luồng nghiệp vụ nhạy cảm liên quan đến mã hóa mật khẩu bằng thuật toán BCrypt, quản lý vòng đời lưu trữ của các Token làm mới bên trong thực thể RefreshTokenRepository.
- user-profile-service: Nơi lưu trữ, cập nhật dữ liệu hồ sơ cá nhân độc lập (Display Name, Bio) và xử lý luồng tải lên tệp tin ảnh đại diện giới hạn dung lượng dưới 2MB tại UserProfileRepository.
- friend-service: Điều khiển toàn bộ đồ thị xã hội cá nhân thông qua việc truy vấn, thiết lập trạng thái kết nối bạn bè (PENDING, ACCEPTED, BLOCKED) trong FriendshipRepository.

2. Thành viên Máy chủ (Server Member)

Tác nhân đại diện cho một Người dùng phổ thông đã hoàn tất chu trình gia nhập vào một không gian Máy chủ cộng đồng (Server) cụ thể:

- server-service: Thực hiện kiểm tra ràng buộc hội viên, tự động thêm thực thể vào bảng quản lý thành viên MemberRepository khi người dùng gửi mã mời hợp lệ.
- messaging-service: Tiếp nhận các luồng dữ liệu thời gian thực được Client đẩy lên thông qua kết nối mạng Socket, kiểm tra chéo trạng thái hội viên ngầm thông qua Feign Client trước khi cấp quyền phân phối tin nhắn văn bản hoặc phương tiện.

3. Quản trị viên Máy chủ (Server Admin)

Tác nhân được Chủ Máy chủ ủy quyền điều hành thông qua việc gán một vai trò cụ thể mang giá trị số nguyên Bitmask đặc quyền được quy định nghiêm ngặt tại hệ thống phân quyền của role-service:

- channel-service: Nơi tiếp nhận các yêu cầu khởi tạo, cấu hình chủ đề (topic), thiết lập chế độ giới hạn tốc độ gửi tin (slowmode), hoặc xóa bỏ các kênh văn bản/kênh thoại trực thuộc.

- role-service & server-service: Thực thi các Use Case mang tính chất điều phối thành viên, so sánh cấp bậc đặc quyền toán tử bit để trục xuất (Kick) hoặc cấm vĩnh viễn (Ban) các tài khoản vi phạm quy chế.

4. Chủ Máy chủ (Server Owner)

Tác nhân khởi tạo Máy chủ, nắm giữ giá trị số nguyên Bitmask tối cao (giá trị 255) được quy định tại lớp lõi của role-service. Sở hữu đặc quyền độc bản tuyệt đối trong việc cấu hình sâu toàn bộ hệ thống vai trò trực thuộc, gán quyền lực quản trị cho các thành viên cấp dưới, thực hiện chuyển nhượng đặc quyền sở hữu tối cao hoặc kích hoạt Use Case giải tán (xóa vĩnh viễn) Máy chủ, tự động kéo theo chu trình dọn dẹp bất đồng bộ toàn bộ hệ thống kênh liên quan.

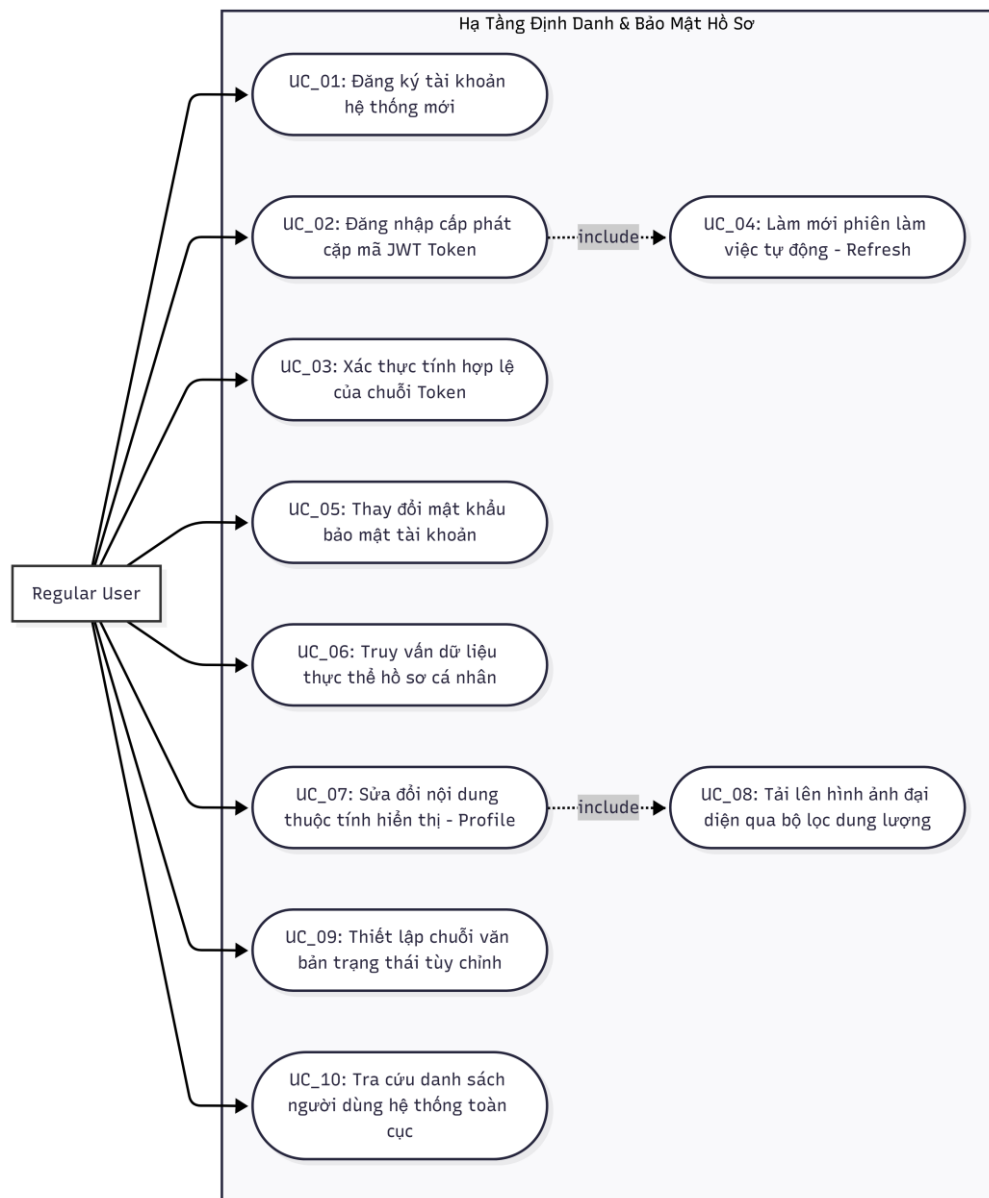
5. Hệ thống tự động / Dịch vụ ngầm (System / Background Worker)

Tác nhân phi nhân loại vận hành liên tục trên hạ tầng máy chủ nhằm bảo đảm tính toàn vẹn và an toàn dữ liệu:

- presence-service: Giám sát luồng giữ kết nối (Heartbeat tầng mạng), tự động phát hiện các sự kiện mất kết nối đột ngột của ứng dụng Client để chuyển dịch trạng thái lưu trữ đệm về OFFLINE, đồng thời đóng gói dữ liệu đẩy lên sàn giao dịch tin nhắn chat.exchange để thông báo bất đồng bộ đến toàn hệ thống.
- log-service: Tiêu thụ (Consume) các thông điệp sự kiện từ hàng đợi RabbitMQ thông qua cấu hình chat.log.queue, tự động bóc tách siêu dữ liệu để ghi vết kiểm toán toàn bộ lịch sử vận hành phân tán dưới định dạng tệp JSON Lines ổn định.
- notification-service: Lắng nghe các sự kiện qua hàng đợi chat.notification.queue, xử lý bóc tách cú pháp tin nhắn để phát hiện các mẫu nhắc tên (@username, @everyone), tính toán bộ đếm tin nhắn chưa đọc tại UnreadCounterRepository và đẩy thông báo thời gian thực xuống Client.

3.3.2 Sơ đồ Use Case Tổng quan Toàn hạ tầng Hệ thống

Sơ đồ tổng quan dưới đây áp dụng kiến trúc luồng chức năng phẳng, kết nối trực tiếp hệ thống tác nhân phân cấp vào ranh giới hệ thống tích hợp 12 dịch vụ lõi của dự án.



Hình 2: Sơ đồ usecase Hạ tầng Định danh và Bảo mật

Các bảng đặc tả chi tiết thuộc Hạ tầng Định danh và Bảo mật:

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_01
Tên Use Case	Đăng ký tài khoản hệ thống mới
Tác nhân chính	Regular User
Dịch vụ phụ trách	gateway-service, auth-service
Tiền điều kiện	Tên đăng nhập yêu cầu chưa từng tồn tại trong hệ thống cơ sở dữ liệu.

Hậu điều kiện	Một bản ghi người dùng mới được khởi tạo thành công tại bảng users.
Luồng xử lý chính	1. Người dùng nhập thông tin Username và Password tại giao diện RegisterPanel của client-app. 2. Ứng dụng gửi gói tin HTTP Post chứa thông tin đăng ký đến API Gateway. 3. Gateway chuyển tiếp yêu cầu đến dịch vụ auth-service. 4. Hệ thống thực hiện kiểm tra chéo thông qua hàm existsByUsername(). Nếu chưa tồn tại, mật khẩu được xử lý mã hóa bằng thuật toán mã hóa BCrypt của lớp SecurityConfig và lưu trực tiếp xuống database thông qua UserRepository.

Bảng 1: Đặc tả Use Case UC_01 - Đăng ký tài khoản hệ thống

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_02
Tên Use Case	Đăng nhập hệ thống & Cấp phát cặp mã JWT Token
Tác nhân chính	Regular User
Dịch vụ phụ trách	gateway-service, auth-service
Tiền điều kiện	Tài khoản và mật khẩu của người dùng đã tồn tại và hoạt động bình thường trên hệ thống.
Hậu điều kiện	Hệ thống sinh và trả về cặp chuỗi mã hóa bao gồm Access Token và Refresh Token cho Client.
Luồng xử lý chính	1. Người dùng tiến hành nhập Username và mật khẩu thông qua bảng điều khiển LoginPanel. 2. Yêu cầu truyền qua API Gateway để định tuyến chính xác vào phân hệ dịch vụ auth-service. 3. AuthService thực hiện truy vấn thực thể người dùng và tiến hành đối sánh mật khẩu thông qua hàm băm passwordEncoder.matches(). 4. Nếu thông tin trùng khớp, hệ thống sử dụng thư viện Jwts.builder() kết hợp khóa bí mật jwtSecret để sinh chuỗi Access Token. Đồng thời tạo ngẫu nhiên một mã UUID để làm Refresh Token lưu trữ lâu dài tại RefreshTokenRepository, rồi phản hồi kết quả về Client.

Bảng 2: Đặc tả Use Case UC_02 - Đăng nhập hệ thống

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_04
Tên Use Case	Làm mới phiên làm việc tự động (Refresh Token)
Tác nhân chính	Regular User (Thực hiện tự động bởi Client)
Dịch vụ phụ trách	gateway-service, auth-service
Tiền điều kiện	Phiên Access Token của người dùng đã hết hạn nhưng Refresh Token lưu trữ tại Client vẫn còn hạn lực.
Hậu điều kiện	Hệ thống cấp phát một chuỗi Access Token mới để Client duy trì kết nối không gián đoạn.
Luồng xử lý chính	1. Ứng dụng client-app phát hiện mã Access Token hiện tại đã hết thời gian hiệu lực khi gửi yêu cầu mạng. 2. Client tự động trích xuất chuỗi Refresh Token đang lưu trữ cục bộ và gửi một HTTP Post Request đến đường dẫn /api/auth/refresh. 3. auth-service tiếp nhận thông tin, gọi lệnh tìm kiếm thực thể mã trong bảng RefreshTokenRepository. 4. Hệ thống kiểm tra thời gian hết hạn thông qua hàm isExpired(). Nếu hợp lệ, hệ thống tái tạo một chuỗi Access Token mới dựa trên thông tin Subject cá nhân và phản hồi về cho Client.

Bảng 3: Đặc tả Use Case UC_04 - Làm mới phiên làm việc tự động

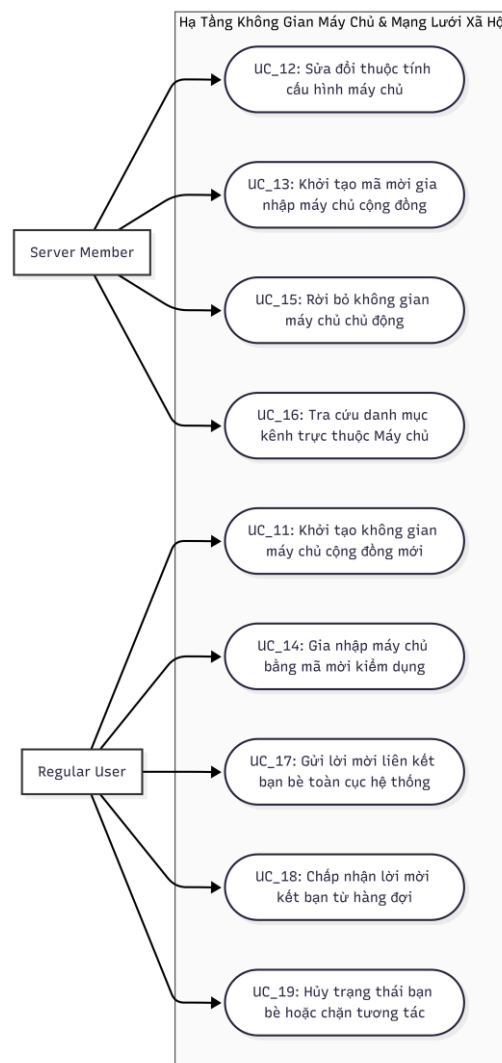
Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_08
Tên Use Case	Tải lên hình ảnh đại diện qua bộ lọc dung lượng
Tác nhân chính	Regular User
Dịch vụ phụ trách	gateway-service, user-profile-service
Tiền điều kiện	Người dùng đã đăng nhập thành công vào hệ thống; tệp tin lựa chọn tải lên có định dạng tệp ảnh hợp lệ (PNG hoặc JPEG).
Hậu điều kiện	Tệp ảnh được lưu trữ thành công trên máy chủ và cập nhật chuỗi đường dẫn URL đại diện mới vào thực thể hồ sơ người dùng.
Luồng xử lý chính	1. Người dùng mở khu vực tùy chỉnh cấu hình tài khoản cá nhân và chọn một tệp hình ảnh từ bộ nhớ máy tính cục bộ.

	2. Ứng dụng khách thực hiện gửi một Multipart HTTP Request chứa tệp tin qua API Gateway để chuyển tiếp thẳng vào phân hệ dịch vụ user-profile-service. 3. Hệ thống kích hoạt bộ lọc kiểm tra dung lượng thuộc lớp GlobalExceptionHandler để bảo đảm kích thước tệp tin không vượt quá ngưỡng quy định 2MB. 4. Nếu tệp tin đáp ứng đầy đủ điều kiện an toàn, hệ thống ghi dữ liệu hình ảnh vào thư mục lưu trữ, gán chuỗi định danh duy nhất UUID để tránh trùng lặp tên tệp, sau đó cập nhật thuộc tính avatarUrl trong bảng dữ liệu UserProfileRepository.
--	---

Bảng 4: Đặc tả Use Case UC_08 - Tải lên hình ảnh hồ sơ, server

2. Nhóm Chức năng Thiết lập Không gian Cộng đồng và Mạng lưới Bạn bè (Server, Channel & Friend Chức năng)

Điều phối toàn bộ các nghiệp vụ liên quan đến việc kiến tạo kết cấu logic của một Máy chủ cộng đồng, phân rã không gian Máy chủ thành các kênh trực thuộc độc lập và quản lý đồ thị xã hội tương tác giữa các tài khoản bạn bè.



Hình 3: Sơ đồ usecase Hạ tầng Server và Bạn bè

Các bảng đặc tả chi tiết thuộc Hạ tầng Không gian Máy chủ và Bạn bè:

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_11
Tên Use Case	Khởi tạo không gian máy chủ cộng đồng mới
Tác nhân chính	Regular User
Dịch vụ phụ trách	server-service, role-service, channel-service
Tiền điều kiện	Người dùng sở hữu tài khoản hợp lệ đang trong trạng thái duy trì kết nối trực tuyến với hệ thống.
Hậu điều kiện	Thực thể Máy chủ được tạo lập; tài khoản khởi tạo được gán quyền tối cao và tự động thiết lập kênh văn bản đầu tiên.
Luồng xử lý chính	1. Người dùng click chọn chức năng thêm máy chủ mới trên bảng điều khiển điều hướng giao diện MiniSidebar của ứng dụng khách. 2. Nhập thông tin tên gọi đại diện và mô tả ngắn của máy chủ tại cửa sổ hộp thoại CreateServerDialog. 3. Phân hệ dịch vụ server-service tiếp nhận yêu cầu mạng, tự động sinh mã mời ngẫu nhiên gồm 8 ký tự bằng hàm <code>UUID.randomUUID().toString().substring(0, 8)</code> và gán giá trị ID tài khoản người tạo vào thuộc tính <code>ownerId</code>. 4. Hệ thống lưu thực thể vào <code>ServerRepository</code>, đồng thời kích hoạt cuộc gọi Feign Client nội bộ sang channel-service để tự động tạo một kênh văn bản mặc định đầu tiên mang tên "General" thuộc quyền sở hữu của Máy chủ nhằm sẵn sàng cho việc giao tiếp.

Bảng 5: Đặc tả Use Case UC_11 - Khởi tạo Server mới

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_14
Tên Use Case	Gia nhập máy chủ bằng mã mời
Tác nhân chính	Regular User
Dịch vụ phụ trách	server-service, role-service

Tiền điều kiện	Chuỗi mã mời do người dùng sử dụng phải chính xác tuyệt đối; tài khoản người dùng không nằm trong danh sách cấm của Máy chủ đó.
Hậu điều kiện	Một bản ghi hội viên mới được thiết lập thành công; tài khoản được cấp quyền đồng bộ cấu trúc kênh để truy cập.
Luồng xử lý chính	1. Người dùng mở bảng điều khiển quản lý gia nhập không gian và điền chuỗi mã mời gồm 8 ký tự vào ô kiểm duyệt hệ thống. 2. server-service tiếp nhận yêu cầu mạng và gọi hàm xử lý dữ liệu <code>findByInviteCode()</code> để truy vấn tìm kiếm thực thể Máy chủ sở hữu chuỗi mã đó. 3. Hệ thống kích hoạt một cuộc gọi Feign Client nội bộ sang phân hệ dịch vụ role-service để chạy hàm kiểm tra <code>checkBanned()</code> nhằm xác minh định danh tài khoản này có đang nằm trong danh sách đen (<code>BannedMemberRepository</code>) hay không. 4. Nếu kết quả kiểm tra trả về trạng thái an toàn, hệ thống khởi tạo một thực thể thành viên <code>Member</code> mới với danh sách mã vai trò <code>roleIds</code> rỗng và thực hiện lưu trữ xuống <code>MemberRepository</code> để hoàn tất chu trình xử lý gia nhập.

Bảng 6: Đặc tả Use Case UC_14 - Gia nhập Server bằng mã mời

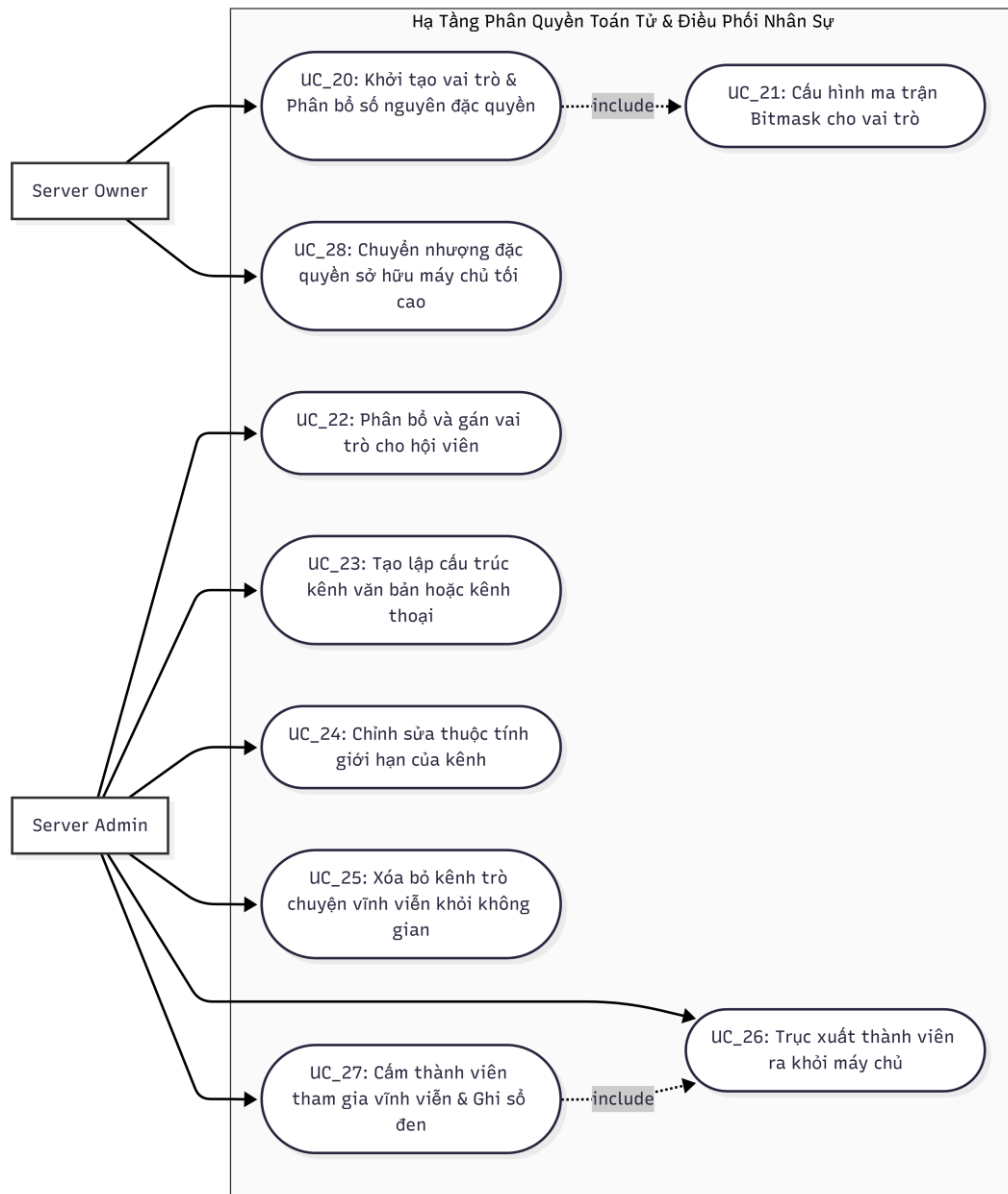
Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_18
Tên Use Case	Chấp nhận lời mời kết bạn từ hàng đợi
Tác nhân chính	Regular User
Dịch vụ phụ trách	friend-service
Tiền điều kiện	Tồn tại một bản ghi liên kết kết bạn ở trạng thái xử lý PENDING do đối phương gửi tới tài khoản từ trước.
Hậu điều kiện	Trạng thái liên kết xã hội giữa hai tài khoản được chuyển dịch chính thức sang thuộc tính hữu hiệu ACCEPTED.

Luồng xử lý chính	1. Người dùng truy cập vào danh mục hiển thị tổng hợp các yêu cầu lời mời kết bạn đang chờ phản hồi trên thành phần giao diện FriendSidebar. 2. Người dùng nhấp chọn nút đồng ý chấp nhận kết bạn đối với tài khoản mong muốn trong danh sách. 3. Yêu cầu thao tác được API Gateway tiếp nhận, kiểm tra chữ ký và định tuyến trực tiếp vào luồng xử lý của phân hệ dịch vụ friend-service. 4. Hệ thống gọi hàm findFriendship() để bóc tách bản ghi dữ liệu tương ứng từ bảng FriendshipRepository, thực hiện thay đổi giá trị thuộc tính trạng thái từ kiểu PENDING thành ACCEPTED, đồng thời cập nhật mốc thời gian hệ thống và đồng bộ lại giao diện.
-------------------	--

Bảng 7: Đặc tả Use Case UC_18 - Chấp nhận lời mời kết bạn từ hàng đợi

3.3.4 Hạ tầng Phân quyền Toán tử Bitmask và Quản trị Quyền hạn Máy chủ (*role-service, server-service, channel-service*)

Hạ tầng này tập trung toàn bộ các chức năng kiểm soát đặc quyền, tính toán ma trận quyền hạn phân tán thông qua các phép toán logic trên dãy bit số nguyên, và thực thi các quyết định điều phối nhân sự, dọn dẹp cấu trúc tài nguyên của hệ thống Máy chủ cộng đồng.



Hình 4: Sơ đồ usecase Hạ tầng Phân quyền và Quản trị

Các bảng đặc tả chi tiết thuộc Hạ tầng Phân quyền và Quản trị:

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_20
Tên Use Case	Khởi tạo vai trò và Phân bổ số nguyên đặc quyền
Tác nhân chính	Server Owner
Dịch vụ phụ trách	role-service
Tiền điều kiện	Tác nhân thực hiện lệnh phải được xác minh là Chủ Máy chủ (ownerId trùng khớp với mã định danh của tài khoản yêu cầu).

Hậu điều kiện	Thực thể vai trò mới được ghi nhận vào hệ thống kèm theo chuỗi giá trị Bitmask cấu hình quyền lực.
Luồng xử lý chính	1. Chủ Máy chủ truy cập vào thành phần giao diện quản trị cấu hình vai trò RoleManagementDialog trên ứng dụng khách. 2. Tiến hành đặt tên vai trò đặc trưng, chọn bảng màu nhận diện và tích chọn các danh mục quyền hạn mong muốn cấp phát. 3. Ứng dụng khách tổng hợp các quyền, tính toán giá trị bit toán tử nhị phân tương ứng và gửi một gói tin HTTP Post chứa siêu dữ liệu vai trò đến role-service. 4. role-service tiếp nhận dữ liệu, thực hiện hàm kiểm tra tính toàn vẹn và thực hiện lưu trữ bản ghi vai trò mới vào bảng dữ liệu thông qua RoleRepository.

Bảng 8: Đặc tả Use Case UC_20 - Khởi tạo vai trò và phân bổ đặc quyền

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_22
Tên Use Case	Phân bổ và gán vai trò cho hội viên máy chủ
Tác nhân chính	Server Admin
Dịch vụ phụ trách	role-service, server-service
Tiền điều kiện	Quản trị viên phải sở hữu Bitmask có chứa quyền quản lý vai trò (MANAGE_ROLES giá trị bit bằng 64); cấp bậc vai trò của người thực hiện phải cao hơn vai trò chuẩn bị gán.
Hậu điều kiện	Dịch vụ cập nhật thành công danh sách mã vai trò roleIds trong hồ sơ thành viên của người dùng đích.

Luồng xử lý chính	1. Quản trị viên kích hoạt hộp thoại AssignRoleDialog bằng cách nhấp chuột phải vào tên một thành viên trong danh sách. 2. Chọn lựa vai trò hợp lệ muốn cấp phát cho thành viên đó từ danh mục hiển thị. 3. Ứng dụng khách gửi yêu cầu REST chứa ID Máy chủ, ID đích và ID vai trò lên phân hệ dịch vụ server-service. 4. server-service thực hiện một cuộc gọi Feign Client nội bộ mạng sang role-service để kiểm tra đặc quyền toán tử bit. Nếu hệ thống xác nhận thao tác hoàn toàn hợp lệ, server-service tiến hành cập nhật mảng dữ liệu roleIds của thực thể thuộc MemberRepository.
-------------------	--

Bảng 9: Đặc tả Use Case UC_22 - Phân bổ và gán vai trò cho hội viên

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_23
Tên Use Case	Tạo lập cấu trúc kênh văn bản hoặc kênh thoại mới
Tác nhân chính	Server Admin
Dịch vụ phụ trách	channel-service, role-service
Tiền điều kiện	Tài khoản yêu cầu phải sở hữu Bitmask chứa đặc quyền tạo kênh (MANAGE_CHANNELS giá trị bit bằng 8 hoặc đặc quyền ADMIN bằng 128).
Hậu điều kiện	Thực thể kênh mới được thiết lập thành công và sẵn sàng tiếp nhận luồng dữ liệu truyền tải hội thoại.

Luồng xử lý chính	1. Quản trị viên mở cửa sổ hộp thoại CreateChannelDialog từ thanh quản trị danh mục cấu trúc kênh của ứng dụng khách. 2. Nhập thông tin chuỗi tên kênh, lựa chọn loại hình kênh (Kênh văn bản - TEXT hoặc Kênh thoại - VOICE) và cấu hình mô tả chủ đề. 3. Ứng dụng khách đẩy gói tin HTTP Post chứa thông tin cấu trúc kênh đến phân hệ dịch vụ đảm nhiệm channel-service. 4. channel-service gọi Feign Client gọi sang role-service để chạy hàm xác thực quyền lực nhị phân. Khi nhận phản hồi thành công, dịch vụ ghi dữ liệu thực thể vào bảng lưu trữ của ChannelRepository và gửi tín hiệu đồng bộ.
-------------------	--

Bảng 10: Đặc tả Use Case UC_23 - Tạo lập cấu trúc kênh văn bản mới

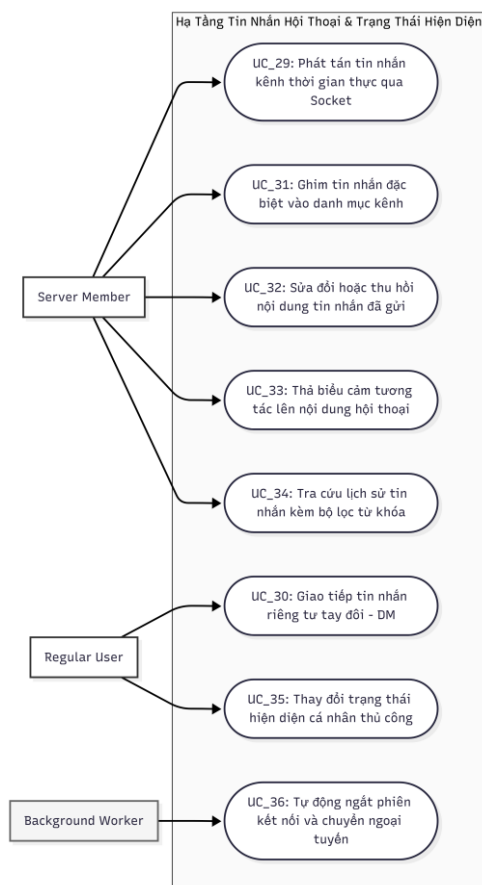
Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_27
Tên Use Case	Cấm thành viên tham gia vĩnh viễn và Ghi sổ đen (Ban)
Tác nhân chính	Server Admin
Dịch vụ phụ trách	role-service, server-service
Tiền điều kiện	Quản trị viên sở hữu vai trò mang giá trị Bitmask chứa quyền cấm thành viên (BAN_MEMBER giá trị bit bằng 32); đối tượng bị xử lý có cấp bậc vai trò thấp hơn người thực hiện.
Hậu điều kiện	Bản ghi thành viên bị xóa khỏi Máy chủ; định danh tài khoản bị áp vào danh sách cấm tại BannedMemberRepository.

Luồng xử lý chính	1. Quản trị viên chọn tính năng cấm thành viên đối với một tài khoản vi phạm thông qua menu ngữ cảnh trên giao diện danh sách hội viên. 2. Ứng dụng khách gửi yêu cầu REST chứa tham số định danh của Máy chủ và hai tài khoản liên quan lên server-service. 3. server-service chuyển tiếp dữ liệu sang role-service để so sánh chéo thứ bậc toán tử bit giữa hai bên. Nếu thỏa mãn điều kiện an toàn, server-service tiến hành xóa liên kết thực thể tại bảng MemberRepository. 4. Đồng thời, role-service thực hiện ghi nhận thông tin tài khoản bị xử lý vào bảng lưu trữ danh sách đen hệ thống của BannedMemberRepository để chặn đứng mọi nỗ lực tham gia lại trong tương lai.
--------------------------	--

Bảng 11: Đặc tả Use Case UC_27 - Cấm thành viên tham gia

3.3.5 Hạ Tầng Hội Thoại Trực Tuyến WebSocket và Truyền Tải Thông điệp Thời Gian Thực (*messaging-service, presence-service, gateway-service*)

Hạ tầng này chịu trách nhiệm điều phối toàn bộ các Use Case liên quan đến luồng truyền tải dữ liệu tin nhắn thời gian thực qua kết nối WebSocket, xử lý tương tác hội thoại, thả biểu cảm, ghim tin nhắn và đồng bộ hóa trạng thái hiện diện của các tài khoản mạng.



Hình 5: Sơ đồ usecase hạ tầng Tin nhắn và Hiện diện

SE330 – Ngôn ngữ lập trình JAVA

Các bảng đặc tả chi tiết thuộc Hạ tầng Tin nhắn và Hiện diện:

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_29
Tên Use Case	Phát tán tin nhắn kênh thời gian thực qua WebSocket Handler
Tác nhân chính	Server Member
Dịch vụ phụ trách	messaging-service, gateway-service
Tiền điều kiện	Thành viên duy trì trạng thái kết nối mạng thông suốt tới WebSocket Endpoint; sở hữu quyền ghi nội dung văn bản vào kênh tương ứng.
Hậu điều kiện	Nội dung tin nhắn hội thoại được lưu giữ ổn định; đồng thời phân phối tức thời đến toàn bộ các tài khoản đang mở kênh qua kết nối Socket.
Luồng xử lý chính	1. Thành viên soạn thảo nội dung văn bản và nhấn nút gửi tại thành phần giao diện nhập liệu ChatInputContainer. 2. Ứng dụng khách đóng gói thông tin cấu trúc thành đối tượng dữ liệu MessageDTO và thực hiện truyền tải qua luồng Socket hiện hành. 3. Thành phần ChatWebSocketHandler thuộc phân hệ dịch vụ messaging-service tiếp nhận gói tin dữ liệu, thực hiện kiểm tra tư cách hội viên nội bộ mạng. 4. Hệ thống ghi dữ liệu tin nhắn vào cơ sở dữ liệu MongoDB/MySQL thông qua MessageRepository, sau đó đẩy sự kiện lên sàn giao dịch tin nhắn trung gian RabbitMQ để kích hoạt luồng đồng bộ giao diện thời gian thực trên toàn bộ ứng dụng khách liên quan.

Bảng 12: Đặc tả Use Case UC_29 - Phát tin nhắn kênh thời gian thực

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_31
Tên Use Case	Ghim tin nhắn đặc biệt vào danh mục kênh hội thoại
Tác nhân chính	Server Member
Dịch vụ phụ trách	channel-service, messaging-service

Tiền điều kiện	Người dùng sở hữu vai trò mang ma trận Bitmask chứa đặc quyền ghim thông tin (MANAGE_MESSAGES giá trị bit bằng 4 hoặc đặc quyền ADMIN bằng 128).
Hậu điều kiện	Thực thể tin nhắn được đánh dấu và bổ sung vào bảng danh sách ghim của PinnedMessageRepository.
Luồng xử lý chính	1. Người dùng click chọn chức năng ghim tin nhắn từ menu tương tác ngữ cảnh của một tin nhắn cụ thể trên bảng hiển thị ChatMessageItem. 2. Ứng dụng khách gửi yêu cầu REST chứa mã định danh tin nhắn và mã định danh kênh lên phân hệ dịch vụ hệ thống channel-service. 3. channel-service tiếp nhận, chạy tiến trình gọi nội bộ xác thực quyền hạn. Nếu hợp lệ, hệ thống tạo lập một bản ghi liên kết mới bên trong bảng dữ liệu PinnedMessageRepository. 4. Hệ thống phát đi một thông điệp sự kiện cập nhật cấu trúc thông qua sản giao dịch RabbitMQ để buộc các ứng dụng khách đang trực tuyến tự động làm mới danh mục hiển thị tại thành phần giao diện PinnedMessagesDialog.

Bảng 13: Đặc tả Use Case UC_31 - Ghim tin nhắn vào danh mục kênh

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_33
Tên Use Case	Thả biểu cảm tương tác lên nội dung hội thoại
Tác nhân chính	Server Member
Dịch vụ phụ trách	messaging-service
Tiền điều kiện	Người dùng duy trì kết nối trực tuyến và có quyền truy cập đọc nội dung bên trong kênh hoặc luồng hội thoại mục tiêu.
Hậu điều kiện	Bản ghi biểu cảm được lưu trữ và cập nhật bộ đếm tương tác hiển thị công khai trên tin nhắn đích.

Luồng xử lý chính	1. Người dùng nhấp chọn biểu tượng cảm xúc mong muốn từ bảng trợ giúp biểu cảm EmojiHelper đặt trên giao diện tin nhắn. 2. Ứng dụng khách gửi gói tin xử lý chứa mã tin nhắn, ID người dùng và ký tự biểu cảm đã chọn đến API Endpoint của phân hệ messaging-service. 3. Dịch vụ tiếp nhận yêu cầu, gọi hàm xử lý lưu trữ dữ liệu thực thể tương tác vào bảng lưu trữ của MessageReactionRepository. 4. Hệ thống đóng gói sự kiện thay đổi và thực hiện phát tán bất đồng bộ qua mạng để cập nhật tức thời số lượng biểu cảm hiển thị dưới chân tin nhắn trên màn hình của toàn bộ các thành viên khác đang xem hội thoại.
-------------------	---

Bảng 14: Đặc tả Use Case UC_33 - Thả biểu cảm tương tác lên tin nhắn

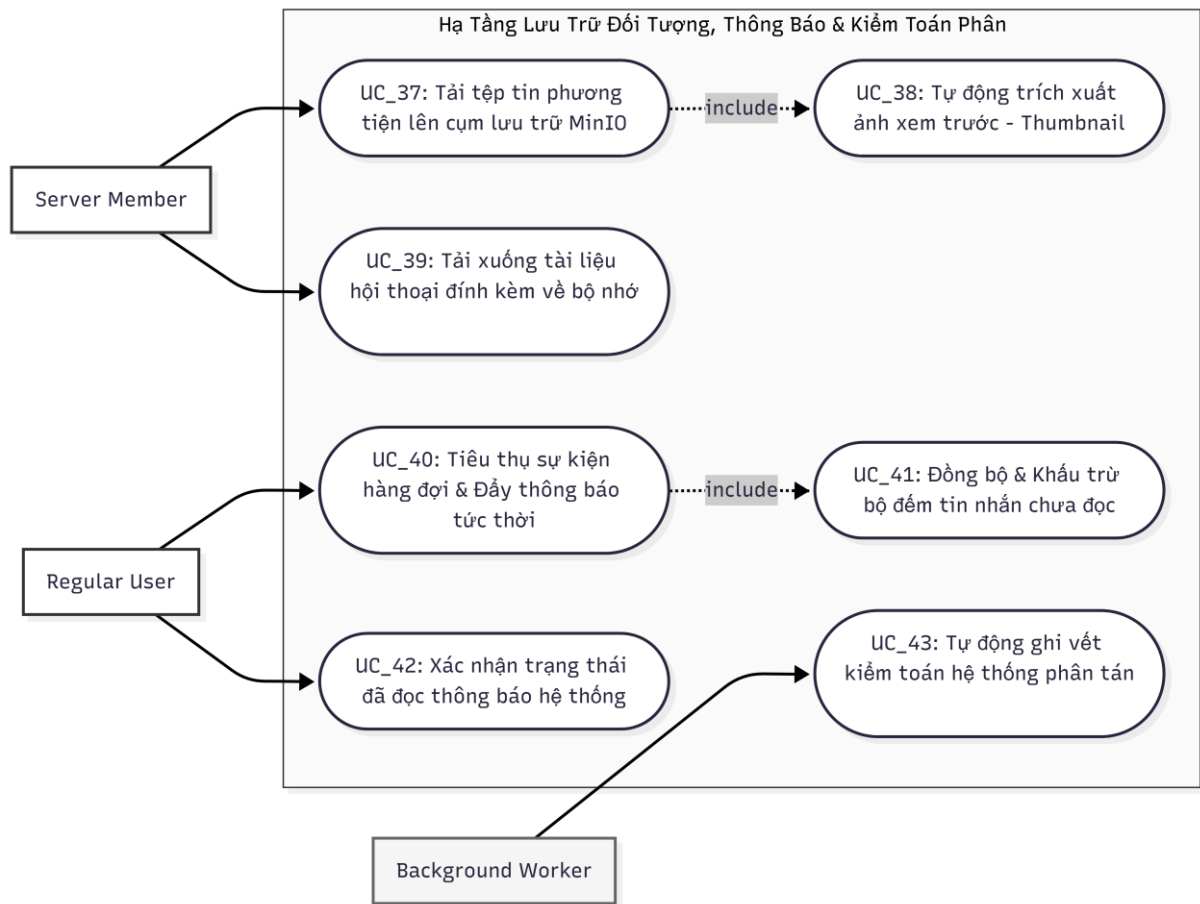
Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_36
Tên Use Case	Tự động ngắt phiên kết nối và chuyển ngoại tuyến (Offline)
Tác nhân chính	Background Worker
Dịch vụ phụ trách	presence-service, messaging-service
Tiền điều kiện	Kết nối mạng vật lý TCP giữa ứng dụng khách và hệ thống bị đứt gãy hoặc Client đột ngột ngừng phản hồi gói tin Heartbeat.
Hậu điều kiện	Trạng thái lưu trữ đệm của tài khoản chuyển về OFFLINE; danh sách bạn bè nhận được thông báo thay đổi.

Luồng xử lý chính	1. Bộ giám sát luồng mạng phát hiện kết nối Socket của một tài khoản cụ thể bị ngắt kết nối đột ngột mà không qua chu trình đóng luồng tiêu chuẩn. 2. Thành phần ChatWebSocketHandler thuộc dịch vụ messaging-service lập tức đón nhận sự kiện ngắt kết nối vật lý và phát tín hiệu báo tin. 3. Hệ thống gửi yêu cầu điều khiển ngầm sang phân hệ dịch vụ presence-service để thay đổi thuộc tính lưu trữ thực thể trạng thái về giá trị số nguyên hoặc chuỗi quy định kiểu ngoại tuyến (OFFLINE). 4. presence-service cập nhật bộ nhớ đệm, gạt bỏ phiên cũ và phát hành một sự kiện trạng thái qua sàn giao dịch chat.exchange với khóa định tuyến quy định để tự động cập nhật biểu tượng trạng thái PresenceStatusIcon trên danh sách bạn bè của đối phương.
-------------------	---

Bảng 15: Đặc tả Use Case UC_36 - Tự động ngắt phiên kết nối và chuyển ngoại tuyến

3.3.6 Hạ Tầng Lưu Trữ Đối Tượng MinIO, Đồng Bộ Thông Báo Đẩy và Kiểm Toán Hệ Thống (**file-service**, **notification-service**, **log-service**)

Hạ tầng phụ trợ chịu trách nhiệm điều tiết toàn bộ các Use Case liên quan đến việc xử lý các tệp tin phương tiện dung lượng lớn thông qua hệ thống lưu trữ đối tượng, phân tích cú pháp tin nhắn để quản lý bộ đếm chưa đọc, điều phối dòng thông báo đẩy thời gian thực và ghi vết log vận hành.



Hình 6: Sơ đồ usecase Hạ tầng Lưu trữ, Thông báo

Các bảng đặc tả chi tiết thuộc Hạ tầng Lưu trữ, Thông báo và Kiểm toán:

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_37
Tên Use Case	Tải tệp tin phương tiện lên cụm lưu trữ MinIO
Tác nhân chính	Server Member
Dịch vụ phụ trách	file-service
Tiền điều kiện	Dung lượng tệp tin đính kèm nằm trong giới hạn cấu hình cho phép của hệ thống (max-file-size quy định trong tệp cấu hình).
Hậu điều kiện	Khối dữ liệu nhị phân được lưu trữ an toàn trong MinIO Bucket; đường dẫn URL tĩnh được ghi nhận vào hệ thống.

Luồng xử lý chính	1. Thành viên nhấn chọn tính năng đính kèm tài liệu phương tiện tại giao diện điều khiển tải tệp FileUploadController trên màn hình chat. 2. Ứng dụng khách tiến hành bấm nhỏ tệp tin và thực hiện gửi một Multipart HTTP Request chứa luồng byte dữ liệu đến phân hệ dịch vụ file-service. 3. file-service tiếp nhận yêu cầu mạng, kích hoạt bộ kiểm tra định dạng và rà soát dung lượng an toàn thuộc lớp cấu hình hệ thống. 4. Hệ thống sinh chuỗi mã khóa bảo mật UUID ngẫu nhiên để đặt tên tệp mới, đẩy trực tiếp luồng dữ liệu nhị phân vào cấu trúc lưu trữ của hệ thống đối tượng MinIO, đồng thời lưu siêu dữ liệu vào bảng của FileMetadataRepository trước khi phản hồi liên kết tĩnh URL về luồng chat.
-------------------	--

Bảng 16: Đặc tả Use Case UC_37 - Tải tệp tin phương tiện lên

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_38
Tên Use Case	Tự động trích xuất ảnh xem trước (Thumbnail)
Tác nhân chính	Server Member (Kích hoạt tự động bởi hệ thống ngầm)
Dịch vụ phụ trách	file-service
Tiền điều kiện	Use Case UC_37 thực hiện tải lên thành công một tệp tin dữ liệu và hệ thống xác định định dạng tệp thuộc nhóm hình ảnh.
Hậu điều kiện	Một tệp ảnh xem trước thu nhỏ dung lượng thấp được khởi tạo và lưu trữ song song bên trong hệ thống MinIO.

Luồng xử lý chính	1. Ngay sau khi luồng xử lý chính của Use Case tải tệp ghi nhận hình ảnh gốc thành công vào hệ thống lưu trữ đối tượng MinIO. 2. Lớp xử lý lỗi ThumbnailService thuộc phân hệ file-service tự động bắt giữ luồng dữ liệu nhị phân của hình ảnh đó trong bộ nhớ tạm. 3. Hệ thống sử dụng thư viện xử lý đồ họa để nén cấu trúc ảnh, giảm độ phân giải xuống kích thước chuẩn xem trước quy định. 4. Thực thể hình ảnh thu nhỏ dung lượng thấp (Thumbnail) được đẩy ngược vào một thư mục lưu trữ đệm riêng biệt của cụm đối tượng MinIO và liên kết đường dẫn URL xem trước này vào thuộc tính hình ảnh của tin nhắn hội thoại để tối ưu băng thông hiển thị cho Client.
-------------------	--

Bảng 17: Đặc tả Use Case UC_38 - Tự động trích xuất ảnh xem trước

Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_40
Tên Use Case	Tiêu thụ sự kiện hàng đợi và Đẩy thông báo tức thời
Tác nhân chính	Regular User
Dịch vụ phụ trách	notification-service
Tiền điều kiện	Xuất hiện một sự kiện tương tác nghiệp vụ đích (tin nhắn mới gửi tới, lượt nhắc tên tài khoản) phát sinh từ các dịch vụ lõi tuyến đầu.
Hậu điều kiện	Thông tin thông báo đẩy được đồng bộ xuống thiết bị và hiển thị trực quan dạng Popup trên màn hình.

Luồng xử lý chính	1. Có hành động tương tác hội thoại mới phát sinh trong hệ thống mạng gây ảnh hưởng trực tiếp đến quyền lợi của một tài khoản cụ thể. 2. Phân hệ dịch vụ notification-service liên tục lắng nghe hàng đợi sự kiện thông qua cấu hình lớp MessageEventListener, tiến hành bắt giữ và tiêu thụ thông điệp từ RabbitMQ. 3. Hệ thống bóc tách siêu dữ liệu, ghi nhận thông tin vào bảng dữ liệu của NotificationRepository, đồng thời tính toán số lượng tin nhắn chưa đọc thông qua việc truy vấn thực thể tại UnreadCounterRepository. 4. Bản ghi thông báo đầy NotificationDTO hoàn chỉnh được truyền trực tiếp xuống Client qua luồng kết nối mạng để kích hoạt hiển thị cửa sổ thông báo trực quan CustomToastNotification.
-------------------	---

Bảng 18: Đặc tả Use Case UC_40 - Tiêu thụ sự kiện hàng đợi và Đẩy thông báo

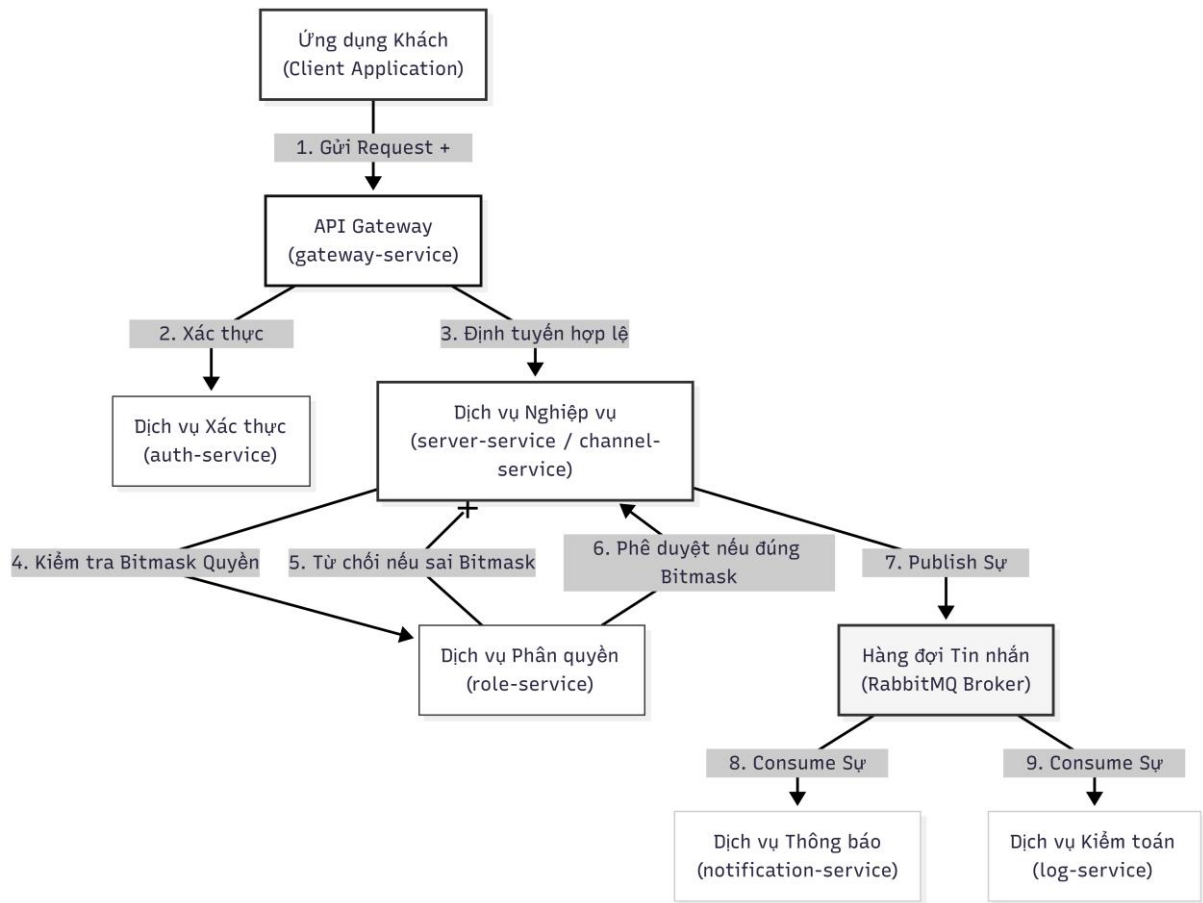
Thành phần đặc tả	Nội dung mô tả chi tiết hệ thống
Mã Use Case	UC_43
Tên Use Case	Tự động ghi vết kiểm toán hệ thống phân tán
Tác nhân chính	Background Worker
Dịch vụ phụ trách	log-service
Tiền điều kiện	Bất kỳ một hành động nghiệp vụ nhạy cảm hoặc thay đổi cấu hình nào được thực thi thành công hoặc thất bại tại 11 phân hệ dịch vụ trước đó.
Hậu điều kiện	Dữ liệu nhật ký vận hành phân tán được cấu trúc hóa và ghi vết lưu trữ lâu dài phục vụ công tác giám sát hệ thống.

Luồng xử lý chính	1. Một dịch vụ thành phần trong hệ thống (ví dụ: auth-service hoặc server-service) hoàn tất chu trình xử lý một gói tin yêu cầu API từ Client. 2. Dịch vụ tự động đóng gói các tham số nghiệp vụ bao gồm định danh dịch vụ, thời gian thực thi, mã định danh tác nhân thao tác và trạng thái phản hồi vào đối tượng chung LogEntry. 3. Đối tượng dữ liệu này được đẩy bất đồng bộ lên sàn giao dịch tin nhắn trung gian RabbitMQ thông qua cấu hình định tuyến quy định. 4. Thành phần lắng nghe sự kiện LogEventListener thuộc phân hệ log-service tiêu thụ thông điệp từ hàng đợi mạng, tiến hành phân tích cú pháp dữ liệu cấu trúc và thực hiện ghi dữ liệu nhật ký vận hành xuống hệ thống tệp tin JSON Lines ổn định.
-------------------	---

Bảng 19: Đặc tả Use Case UC_43 - Tự động ghi log hệ thống phân tán

3.3.7 Ma Trận Ràng Buộc Kiến Trúc và Chu Trình Phối Hợp Liên Dịch Vụ

Để cụ thể hóa cách thức 12 dịch vụ thành phần phối hợp xử lý các chốt chặn an toàn, đặc quyền Bitmask và truyền tải sự kiện hướng cấu trúc, chu trình tương tác toàn cục được mô hình hóa qua sơ đồ luồng phẳng dưới đây:



Hình 7: Chu trình phối hợp các nghiệp vụ

Để bảo đảm tính toàn vẹn dữ liệu và an toàn bảo mật, toàn bộ hệ thống vận hành dựa trên ma trận ràng buộc kỹ thuật nghiêm ngặt. Dưới đây là bảng đặc tả chi tiết các chốt chặn phối hợp:

Thuộc tính ma trận ràng buộc	Cơ chế thiết lập và Phối hợp liên dịch vụ
Tên cơ chế ràng buộc	Chốt chặn Định danh Biên tập trung (Unified Security Gate)
Dịch vụ trực tiếp phụ trách	gateway-service, auth-service
Bản chất kỹ thuật áp dụng	Kiểm soát Token tập trung qua bộ lọc toàn cục JwtAuthFilter.
Chu trình xử lý chi tiết	1. Mọi yêu cầu API từ Client nhắm tới các Use Case nghiệp vụ bắt buộc phải đi qua cổng duy nhất là gateway-service. 2. Bộ lọc JwtAuthFilter bóc tách chuỗi mã hóa JWT tại trường Header và chuyển tiếp nội bộ sang auth-service để kiểm tra tính toàn vẹn. 3. Nếu Token hợp lệ, gateway-service tự động giải mã thông tin tài khoản, bổ sung các thẻ định danh X-User-Id vào gói tin và chuyển tiếp xuống các dịch vụ tuyến sau.

	4. Nếu Token hết hạn hoặc sai chữ ký, hệ thống lập tức chặn đứng, hủy bỏ yêu cầu mạng nghiệp vụ và phản hồi mã lỗi 401 Unauthorized về Client mà không làm tổn tài nguyên xử lý của các dịch vụ vùng lõi.
--	---

Bảng 20: Chi tiết ma trận ràng buộc Định danh

Thuộc tính ma trận ràng buộc	Cơ chế thiết lập và Phối hợp liên dịch vụ
Tên cơ chế ràng buộc	Kiểm soát Đặc quyền Phân tán qua Toán tử Bit (Bitmask RBAC)
Dịch vụ trực tiếp phụ trách	role-service, server-service, channel-service
Bản chất kỹ thuật áp dụng	Tính toán ma trận đặc quyền phân tán thông qua logic nhị phân tốc độ cao.
Chu trình xử lý chi tiết	1. Khi một tác nhân Quản trị viên kích hoạt các Use Case nhạy cảm như xóa kênh, ghim tin hoặc trục xuất thành viên, dịch vụ tiếp nhận (channel-service hoặc server-service) sẽ tạm giữ luồng xử lý. 2. Dịch vụ tuyến đầu thực hiện một cuộc gọi Feign Client đồng bộ kết nối trực tiếp sang phân hệ dữ liệu của role-service. 3. role-service truy vấn bảng vai trò của người dùng, lấy ra chuỗi số nguyên đại diện cho đặc quyền và thực hiện phép toán logic bit AND (&) giữa ma trận quyền hiện tại với mã bit quy định của hành động. 4. Nếu kết quả trả về thỏa mãn điều kiện logic nhị phân, role-service phản hồi trạng thái hợp lệ để dịch vụ tuyến đầu ghi nhận dữ liệu xuống database. Ngược lại, hệ thống lập tức ném ra ngoại lệ RuntimeException để hủy bỏ toàn bộ chu trình thao tác dữ liệu.

Bảng 21: Chi tiết ma trận ràng buộc Kiểm soát Đặc quyền

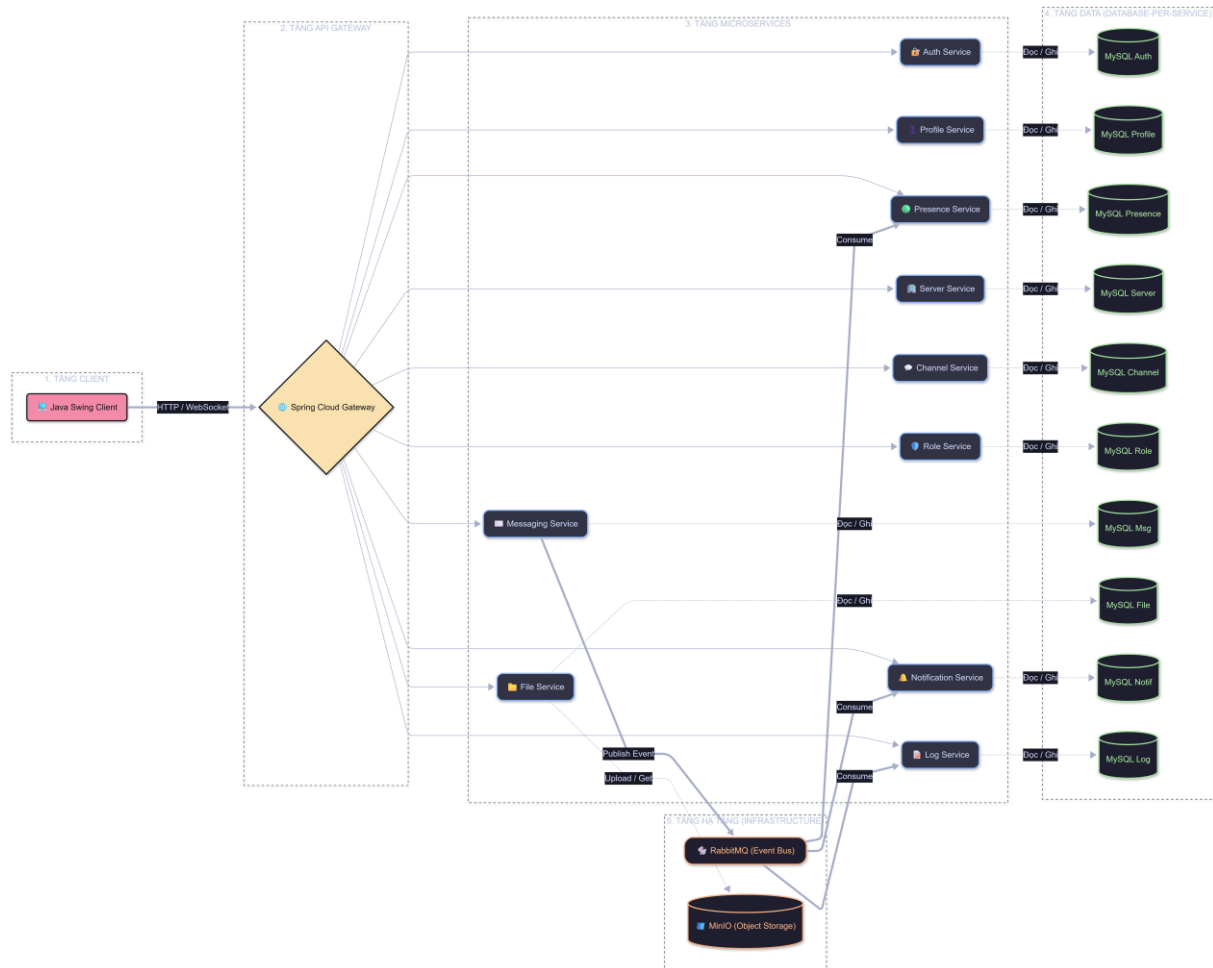
Thuộc tính ma trận ràng buộc	Cơ chế thiết lập và Phối hợp liên dịch vụ
Tên cơ chế ràng buộc	Nhất quán Hướng Sự kiện Bất đồng bộ (Event-Driven Consistency)
Dịch vụ trực tiếp phụ trách	messaging-service, notification-service, log-service
Bản chất kỹ thuật áp dụng	Tách biệt vòng đời xử lý và phân tải hệ thống thông qua hàng đợi RabbitMQ.
Chu trình xử lý chi tiết	1. Đối với các Use Case có tần suất dữ liệu lớn và liên tục như gửi tin nhắn kênh thời gian thực hoặc ngắt kết nối mạng

	đột ngột, hệ thống loại bỏ hoàn toàn cơ chế khóa dữ liệu đồng bộ. 2. Khi messaging-service hoặc presence-service hoàn tất việc cập nhật trạng thái dữ liệu vào database cục bộ, hệ thống ngay lập tức đóng gói thông tin thành các cấu trúc Event chuẩn hóa và đẩy lên sàn giao dịch chat.exchange của RabbitMQ. 3. Hệ thống hàng đợi mạng tự động phân phối bản tin về các hàng đợi độc lập bao gồm chat.notification.queue và chat.log.queue. 4. Tại hạ tầng phụ trợ, notification-service và log-service tự động tiêu thụ sự kiện một cách bất đồng bộ để thực hiện việc cập nhật bộ đếm unread và ghi vết kiểm toán xuống tệp JSON Lines ổn định mà không gây ảnh hưởng đến hiệu năng thời gian thực của luồng chat chính.
--	--

Bảng 22: Chi tiết ma trận ràng buộc Nhất quán Hướng sự kiện Bất đồng bộ

Chương 4: THIẾT KẾ HỆ THỐNG

4.1 Sơ đồ kiến trúc tổng quan hệ thống:

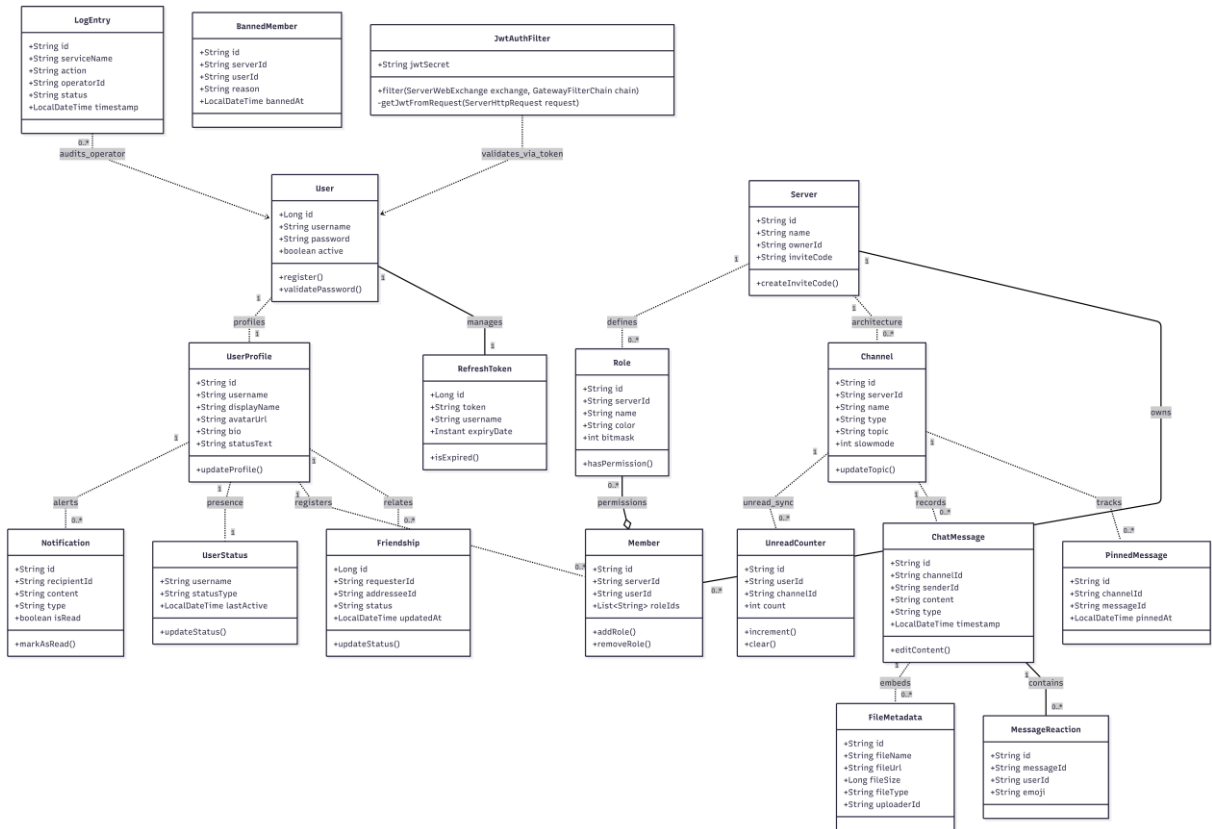


Hình 8: Sơ đồ kiến trúc tổng quan hệ thống

4.2 Sơ đồ lớp:

Sơ đồ lớp chi tiết dưới đây đặc tả cấu trúc thực thể (Domain Entities) được triển khai độc lập trong các lớp mã nguồn Java Spring Boot của các dịch vụ cốt lõi bao gồm **auth-service**, **user-profile-service**, **server-service**, **channel-service**, **role-service**, và **messaging-service**. Sơ đồ phân định rõ ranh giới của từng phân hệ dịch vụ nhưng vẫn thể hiện mối quan hệ logic thông qua các trường định danh ID liên kết chéo.

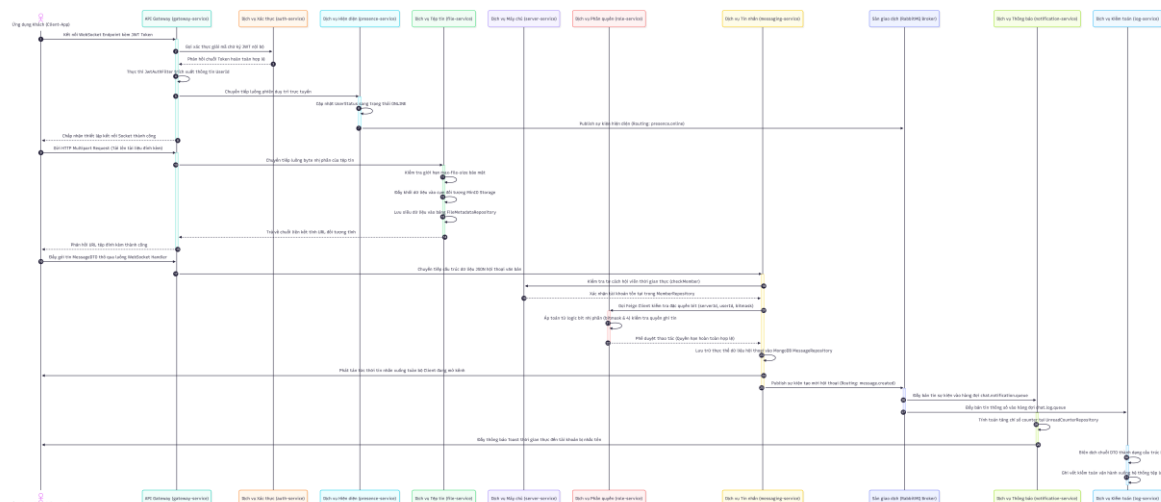
SE330 – Ngôn ngữ lập trình JAVA



Hình 9: Sơ đồ lớp

4.3 Sơ đồ tuần tự cho các luồng liên dịch vụ

Sơ đồ tuần tự dưới đây mô tả chi tiết kịch bản phức tạp nhất hệ thống: **Thành viên gửi tin nhắn thời gian thực vào kênh chung kèm theo đính kèm tài liệu, hệ thống xử lý xác thực bảo mật Gateway, kiểm tra toán tử Bitmask phân quyền đồng bộ và đẩy thông báo bất đồng bộ qua RabbitMQ.**



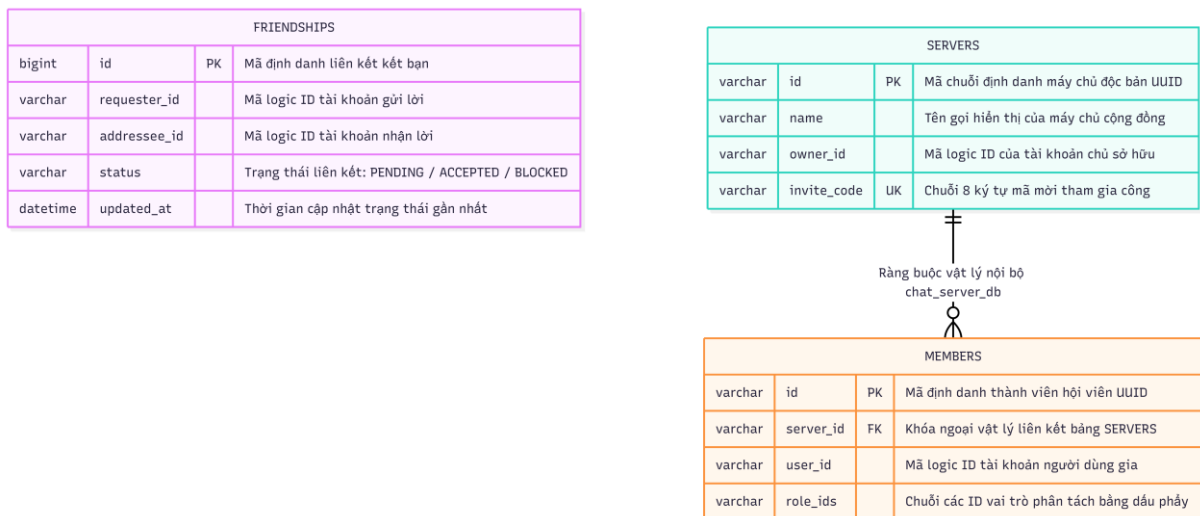
Hình 10: Sơ đồ tuần tự các luồng dịch vụ

4.4 Thiết kế cơ sở dữ liệu chi tiết

Hệ thống áp dụng mô hình cơ sở dữ liệu phân tán (Database per Service) để bảo đảm tính độc lập. Sơ đồ thực thể quan hệ cơ sở dữ liệu tổng hợp dưới đây mô tả chi tiết cấu trúc các bảng dữ liệu được chuẩn hóa, phân tách rõ ràng các liên kết khóa ngoại vật lý và các liên kết khóa logic bằng chuỗi ID giữa các phân hệ quản lý:

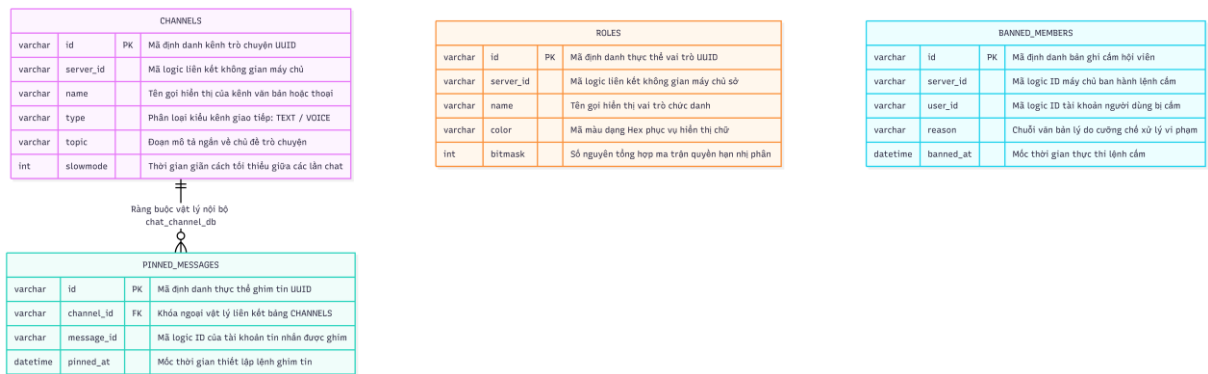


Hình 11: Cơ sở dữ liệu chat_auth_db và chat_profile_db

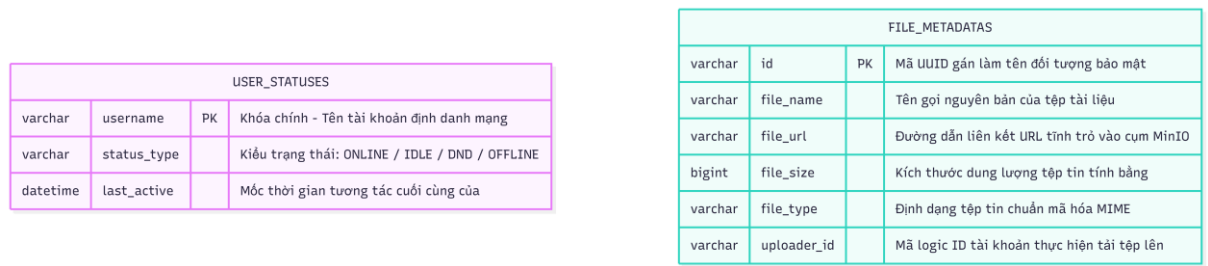


Hình 12: Cơ sở dữ liệu chat_friend_db và chat_server_db

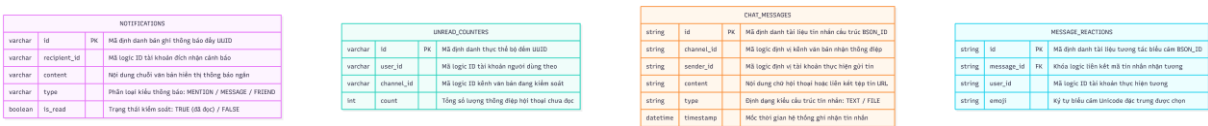
SE330 – Ngôn ngữ lập trình JAVA



Hình 13: Cơ sở dữ liệu chat_channel_db và chat_role_db



Hình 14: Cơ sở dữ liệu chat_presence_db và chat_file_db



Hình 15: Cơ sở dữ liệu chat_notification_db và chat_messaging_db